



ค่ายโอลิมปิกวิชาการวิชาคอมพิวเตอร์ ค่าย 2
ศูนย์มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

กราฟชนิดถ่วงน้ำหนักและอัลกอริทึมกราฟ (Weighted graph & algorithms)

อ.ลือพล พิพานเมฆาภรณ์

luepol.p@sci.kmutnb.ac.th

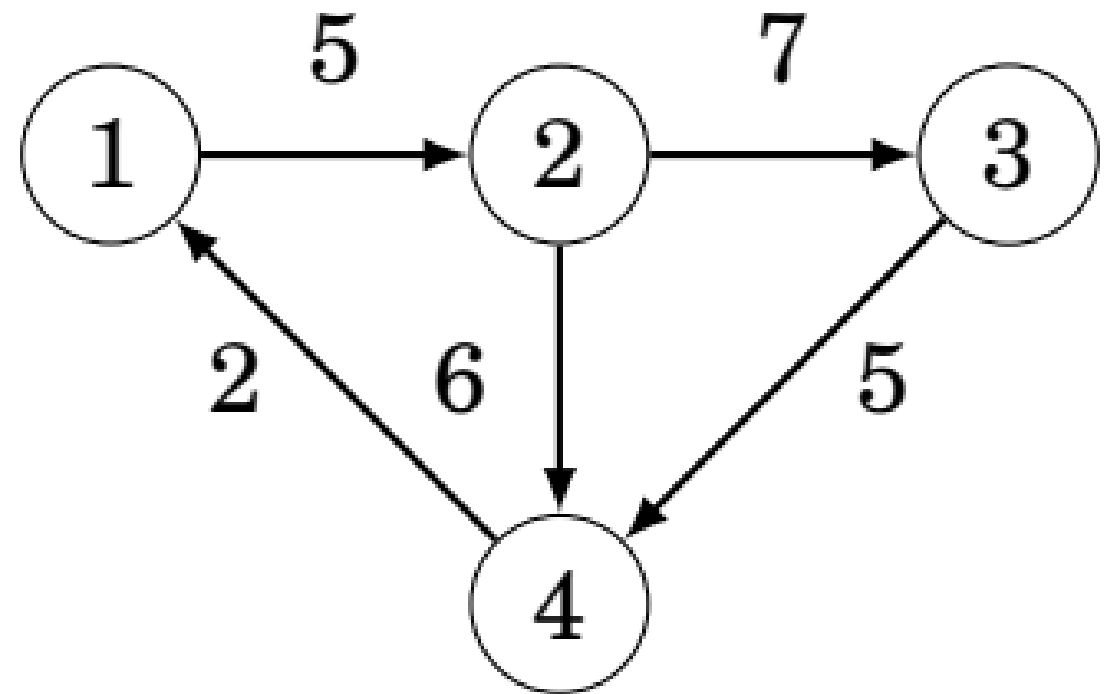
ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

- กราฟชนิดถ่วงน้ำหนัก (Weighted graph)
 - เมตริกซ์ประชิด (Adjacency matrix)
 - ลิสต์ประชิด (Adjacency list)
- ปัญหาการหาเส้นทางสั้นที่สุด (Shortest path) ในกราฟ
 - Single-source shortest path problem
 - All-pair shortest path problem
- ปัญหาต้นไม้แผ่ทั่วที่เล็กที่สุด (Minimum spanning tree) ในกราฟ
 - Prim's algorithm
 - Kruskal's algorithm

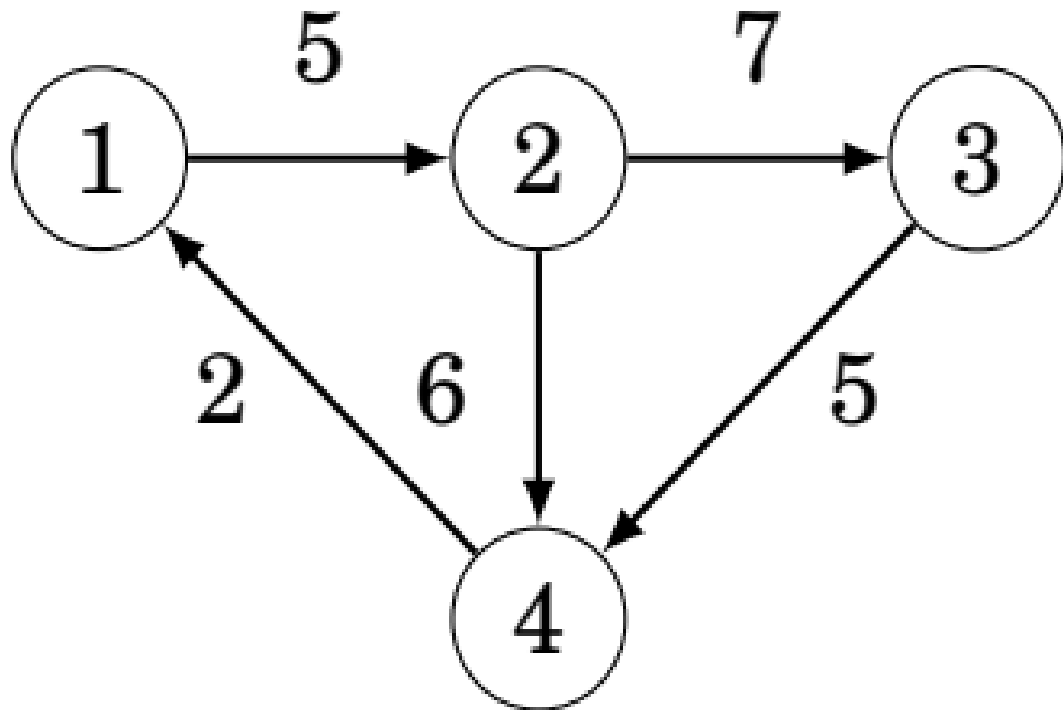
กราฟชนิดถ่วงน้ำหนัก (Weighted graph)

- กราฟชนิดถ่วงน้ำหนัก (weighted graph) เป็นรูปแบบกราฟที่มีการระบุค่าน้ำหนัก (weight) ให้กับเอดจ์ในกราฟ
- ค่าน้ำหนักอาจแทนด้วยระยะทาง (distance) คะแนน (score) เวลา (time) หรือเงื่อนไขอื่นขึ้นอยู่กับการใช้งานกราฟ
- การแสดงกราฟชนิดถ่วงน้ำหนักทำได้ 2 แบบ ได้แก่
 - เมตริกซ์ประชิด (adjacency matrix)
 - ลิสต์ประชิด (adjacency list)



เมตริกซ์ประชิด (Adjacency matrix)

```
vector<vector<int>> graph( n_vertex, vector<int>(n_vertex, 0) );
```



เวอร์เท็กซ์
เริ่มต้น

1
2
3
4

	1	2	3	4
1	0	5	0	0
2	0	0	7	6
3	0	0	0	5
4	2	0	0	0

เวอร์เท็กซ์
ปลายทาง

ค่าน้ำหนัก

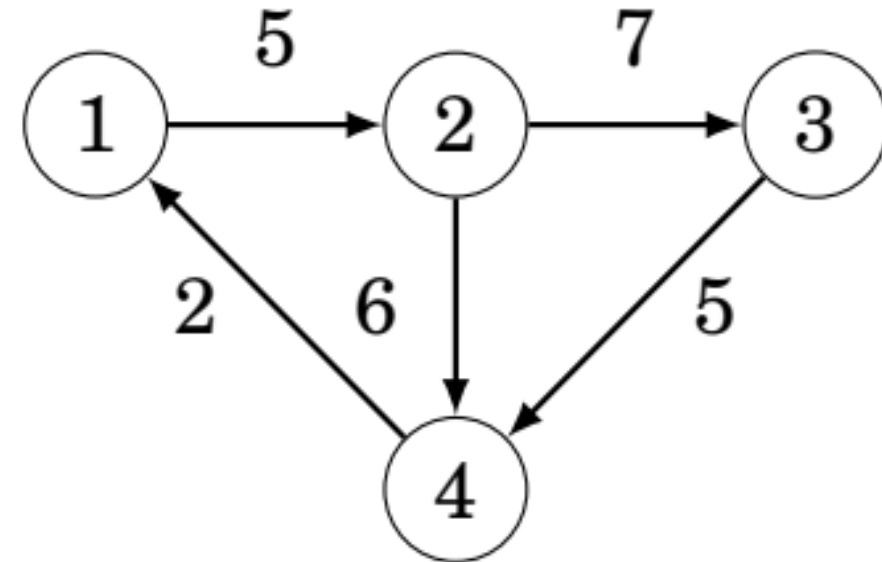
การเข้าถึงสมาชิกในเมตริกซ์ประชิด

```
#include <iostream>
#include <vector>

using namespace std;
int main() {
    int n_vertex = 4;
    vector<vector<int>> graph(n_vertex, vector<int>(n_vertex, 0));
    int numVertices = graph.size();

    graph[0][1] = 5; graph[1][2] = 7; graph[1][3] = 6;
    graph[2][3] = 5; graph[3][0] = 2;

    for(int i=0; i < graph.size(); i++)
    { cout << i << " : ";
      for(int j=0; j < graph[i].size(); j++)
          cout << " " << graph[i][j];
      cout<<endl;
    }
    return 0;
}
```



0	5	0	0
0	0	7	6
0	0	0	5
2	0	0	0

ลิสต์ประชิด (Adjacency list)

- สร้างอาร์เรย์ของเวกเตอร์ (vector) โดยสมาชิกแต่ละตัวจะแสดงในรูปแบบของคู่ลำดับ `pair<int, int>`
 - `pair.first` แทนเวอร์เท็กซ์ปลายทาง
 - `pair.second` แทนค่าน้ำหนัก (weight) ระหว่างเวอร์เท็กซ์

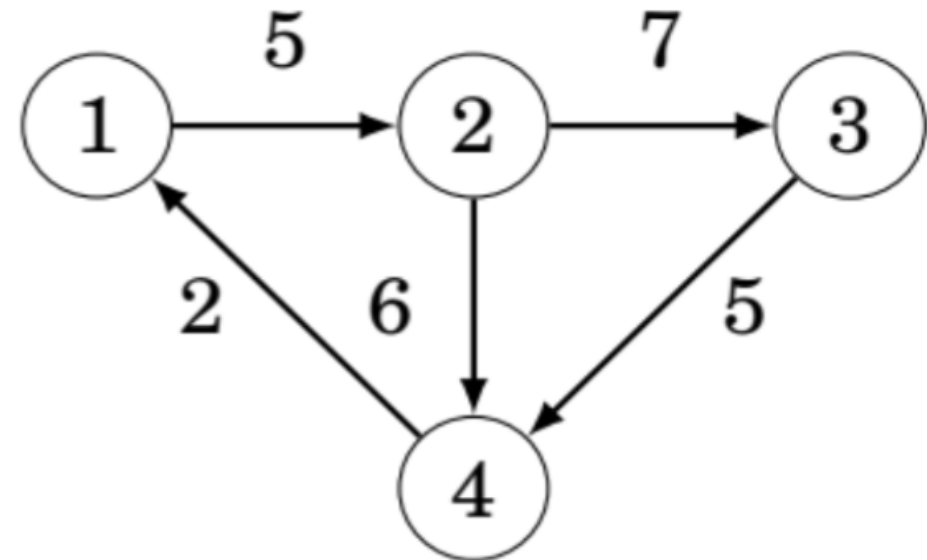
```
vector<pair<int,int>> adj[N];
```

<เวอร์เท็กซ์ปลายทาง, ค่าน้ำหนัก>

```
int main()
{
    vector<pair<int, int> > adj[4];

    adj[1].push_back({2, 5});
    adj[2].push_back({3, 7});
    adj[2].push_back({4, 6});
    adj[3].push_back({4, 5});
    adj[4].push_back({1, 2});

    return 0;
}
```



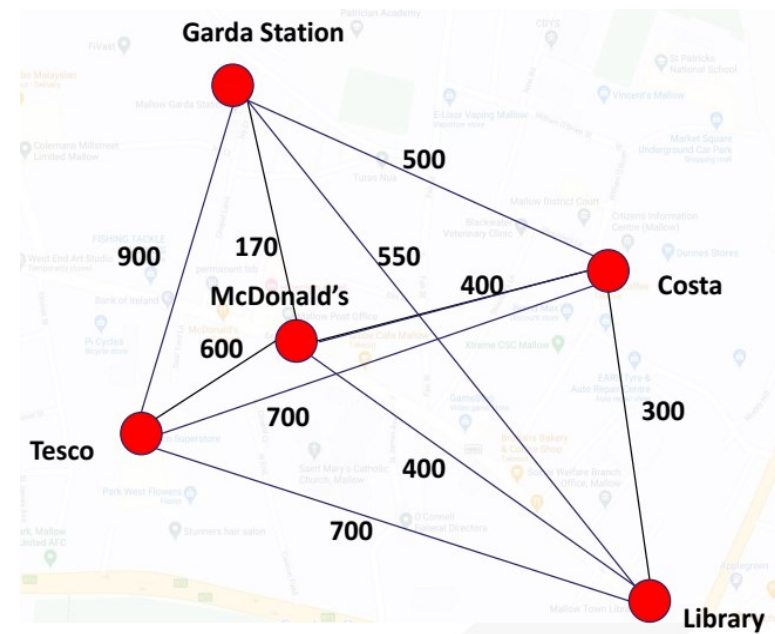
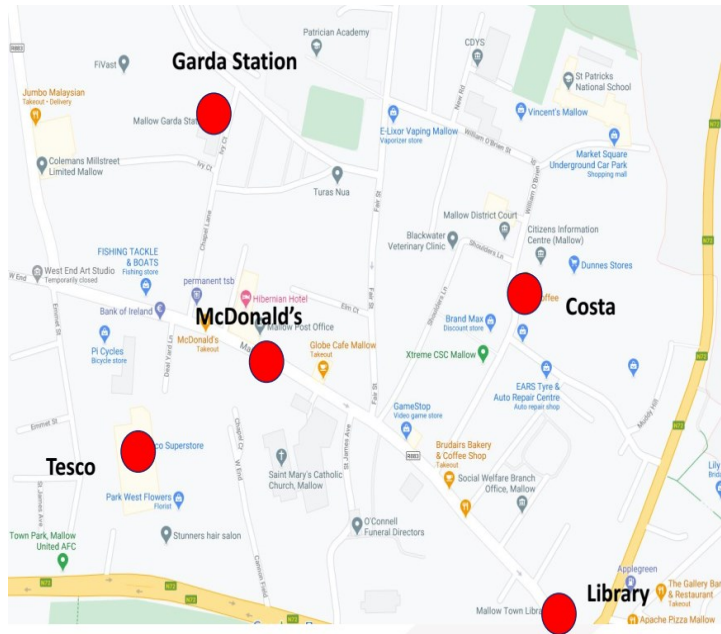
การเข้าถึงสมาชิกในลิสต์ประชิด

```
void print_graph(vector<pair<int, int>> adj[], int V)
{
    for(int i= 1; i <= V; ++i) {
        cout << i << ": ";
        for (int j = 0; j < adj[i].size(); ++j)
            cout << " (" << adj[i][j].first << ", " << adj[i][j].second << ") ";
        cout << "\n";
    }
}
```

```
1:  (2,5)
2:  (3,7) (4,6)
3:  (4,5)
4:  (1,2)
```

ปัญหาเส้นทางที่สั้นที่สุด (Shortest path) ในกราฟ

- เส้นทางสั้นที่สุด (shortest path) หมายถึงเส้นทางระหว่างสองเวอร์เท็กซ์ในกราฟที่มีความยาวรวมของค่าน้ำหนักของเอจด์ (edge) น้อยที่สุด
- ค่าน้ำหนัก (weight) ของเอจด์ แทนระยะทาง (distance) หรือค่าใช้จ่าย (cost) ระหว่างเวอร์เท็กซ์





ปัญหาเส้นทางที่สั้นที่สุด (Shortest path) ในกราฟ

สามารถแบ่งออกปัญหาออกเป็น 2 ประเภท ได้แก่

1. Single Source Shortest Path (SSSP)

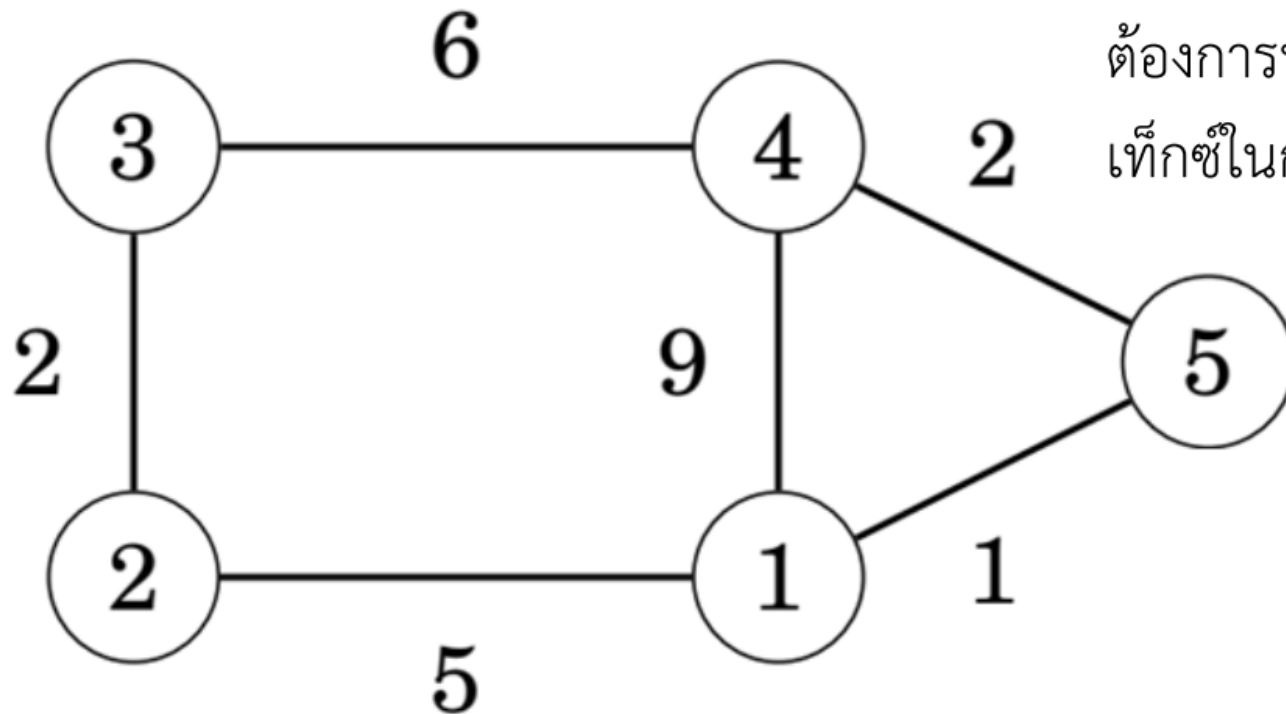
อัลกอริทึม Dijkstra ต้องกำหนดจุดเริ่มต้น

2. All-Pair Shortest Path (APSP)

อัลกอริทึม Floyd-Warshall เริ่มต้นจากจุดใดก็ได้ในกราฟ

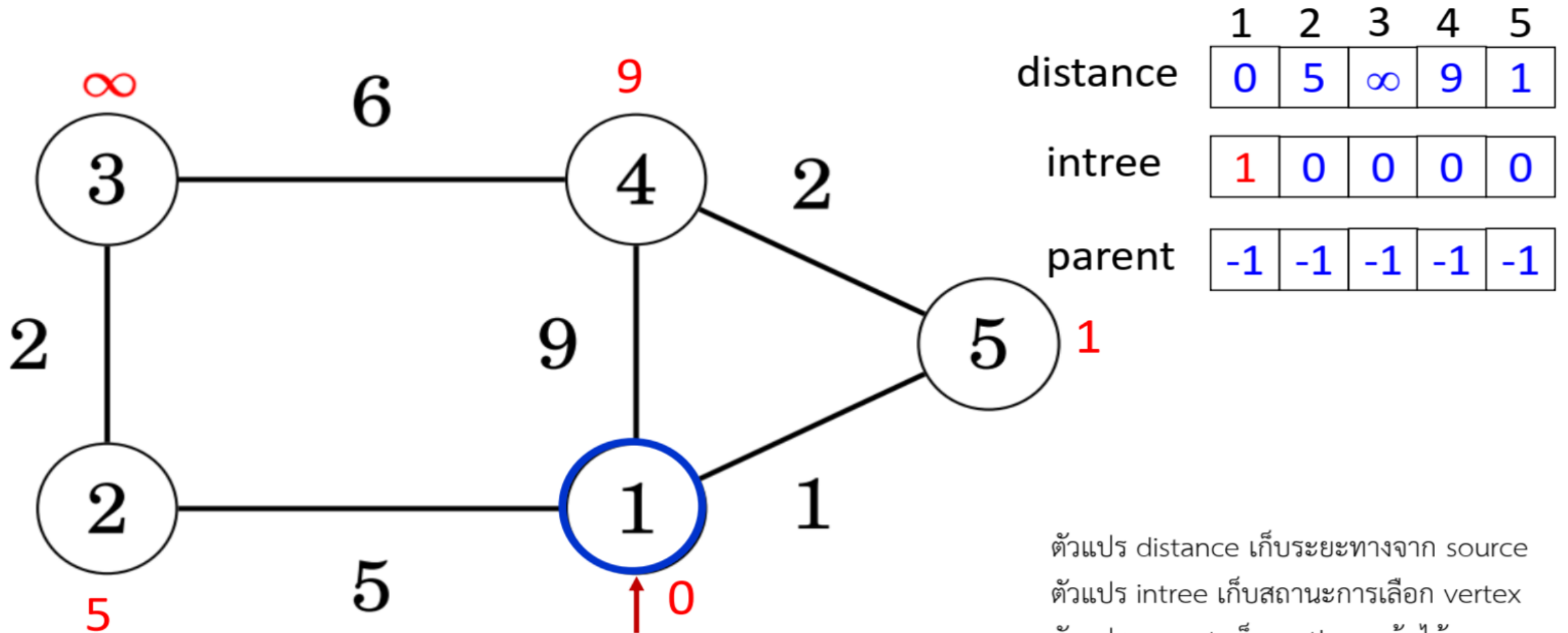
อัลกอริทึม Dijkstra

กำหนดจุดเริ่มต้น (**source vertex**) ในกราฟ เริ่มสร้างต้นไม้ (tree) โดยแผ่ไปยังเอดจ์ (edge) รอบข้าง โดยมีเงื่อนไขว่าเอดจ์ที่จะถูกเลือกเป็นคำตอบจะต้องมีระยะทางจาก source vertex รวมกับน้ำหนักของเอดจ์ดังกล่าวมีค่าน้อยที่สุด ทำซ้ำจนกระทั่งต้นไม้ครอบคลุมครบทุกเวอร์เท็กซ์ในกราฟ



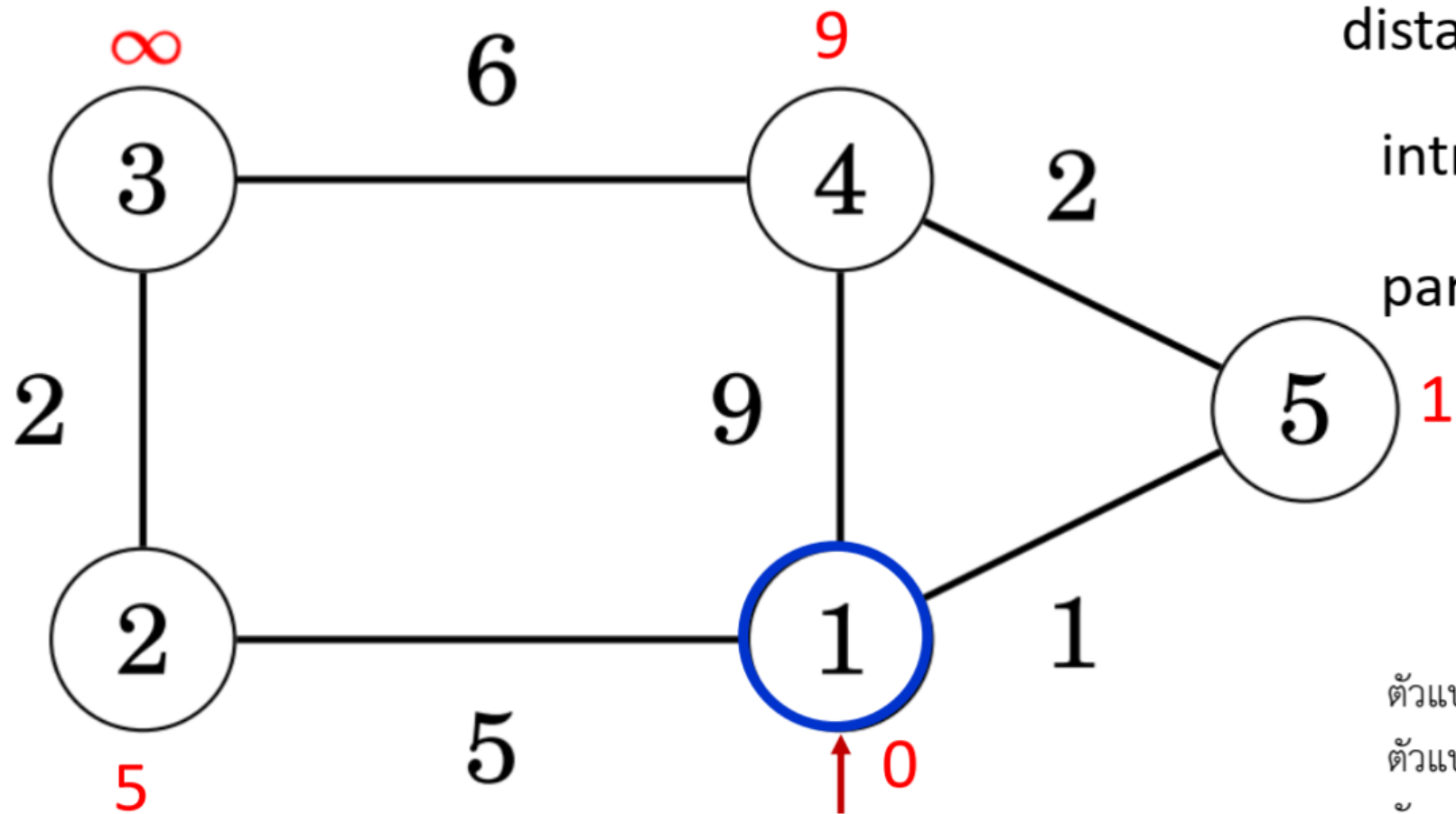
ต้องการหาเส้นทางสั้นสุดจากเวอร์เท็กซ์ **1** ไปยังทุกเวอร์เท็กซ์ในกราฟ

อัลกอริทึม Dijkstra





อัลกอริทึม Dijkstra



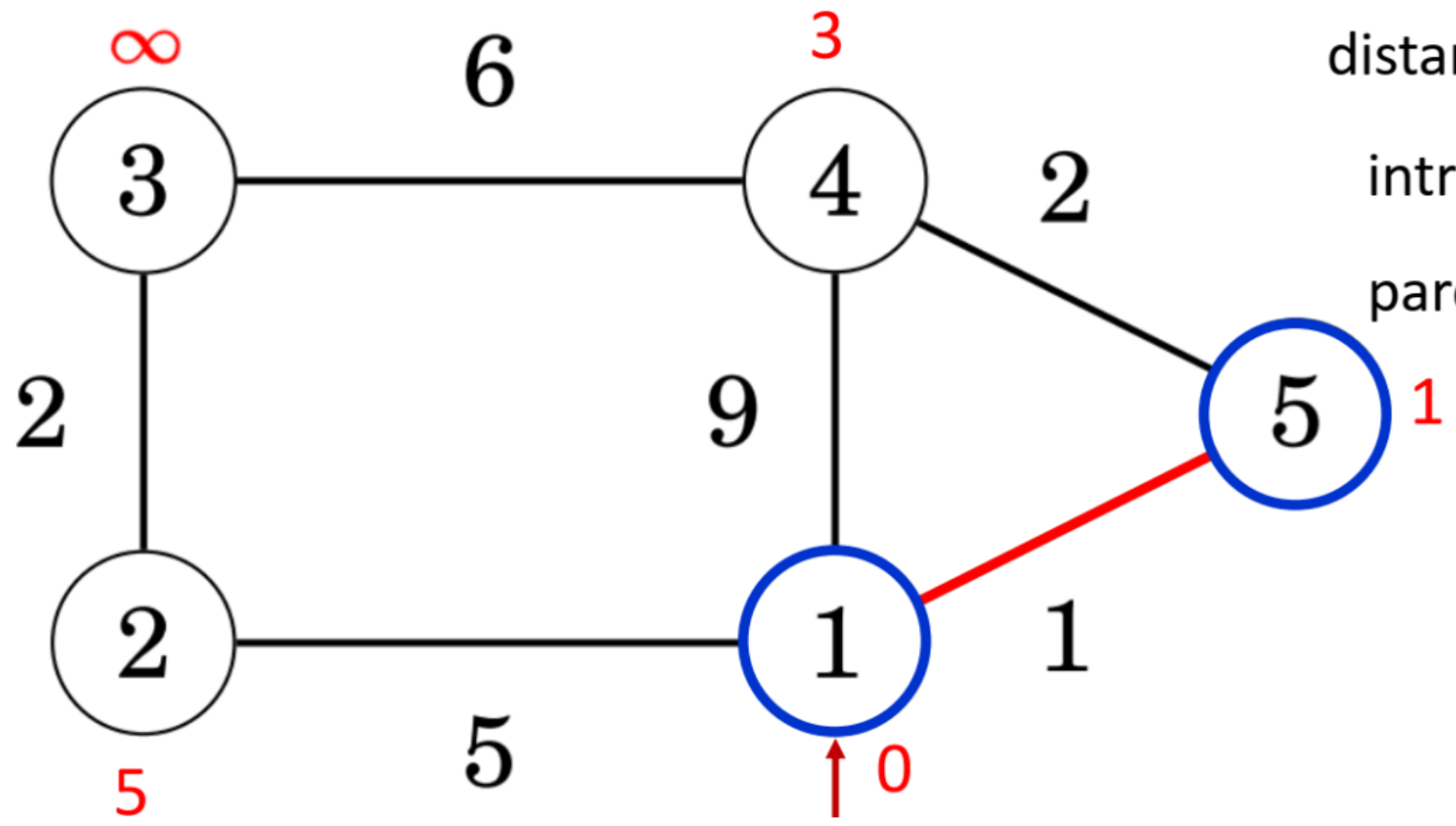
เลือก Vertex ระยะทางน้อยที่สุด

	1	2	3	4	5
distance	0	5	∞	9	1
intree	1	0	0	0	0
parent	-1	-1	-1	-1	-1

ตัวแปร distance เก็บระยะทางจาก source
ตัวแปร intree เก็บสถานะการเลือก vertex
ตัวแปร parent เก็บ path ของต้นไม้



Dijkstra algorithm

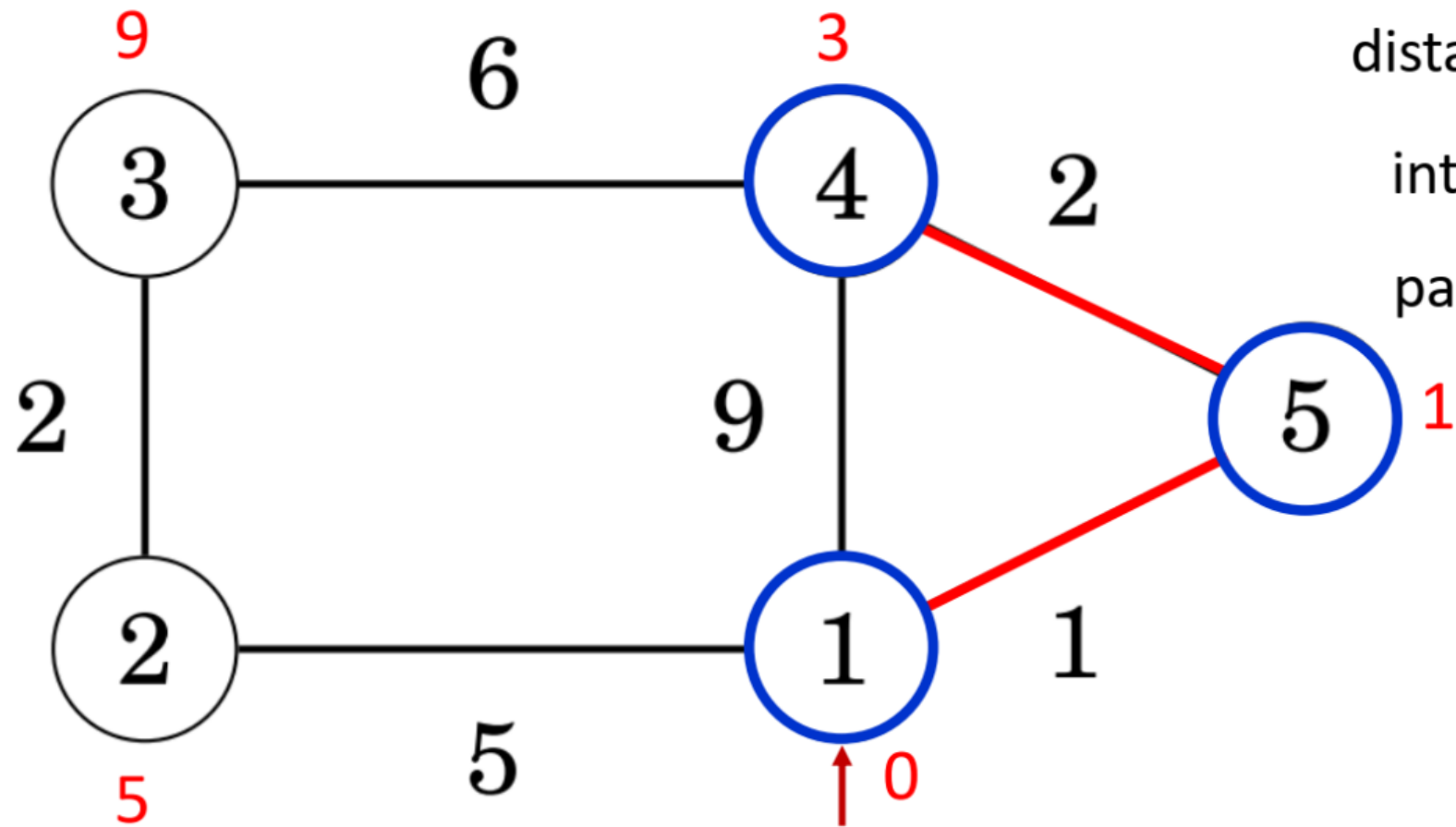


อัปเดตระยะทาง
จาก source

	1	2	3	4	5
distance	0	5	∞	3	1
intree	1	0	0	0	1
parent	-1	-1	-1	-1	1



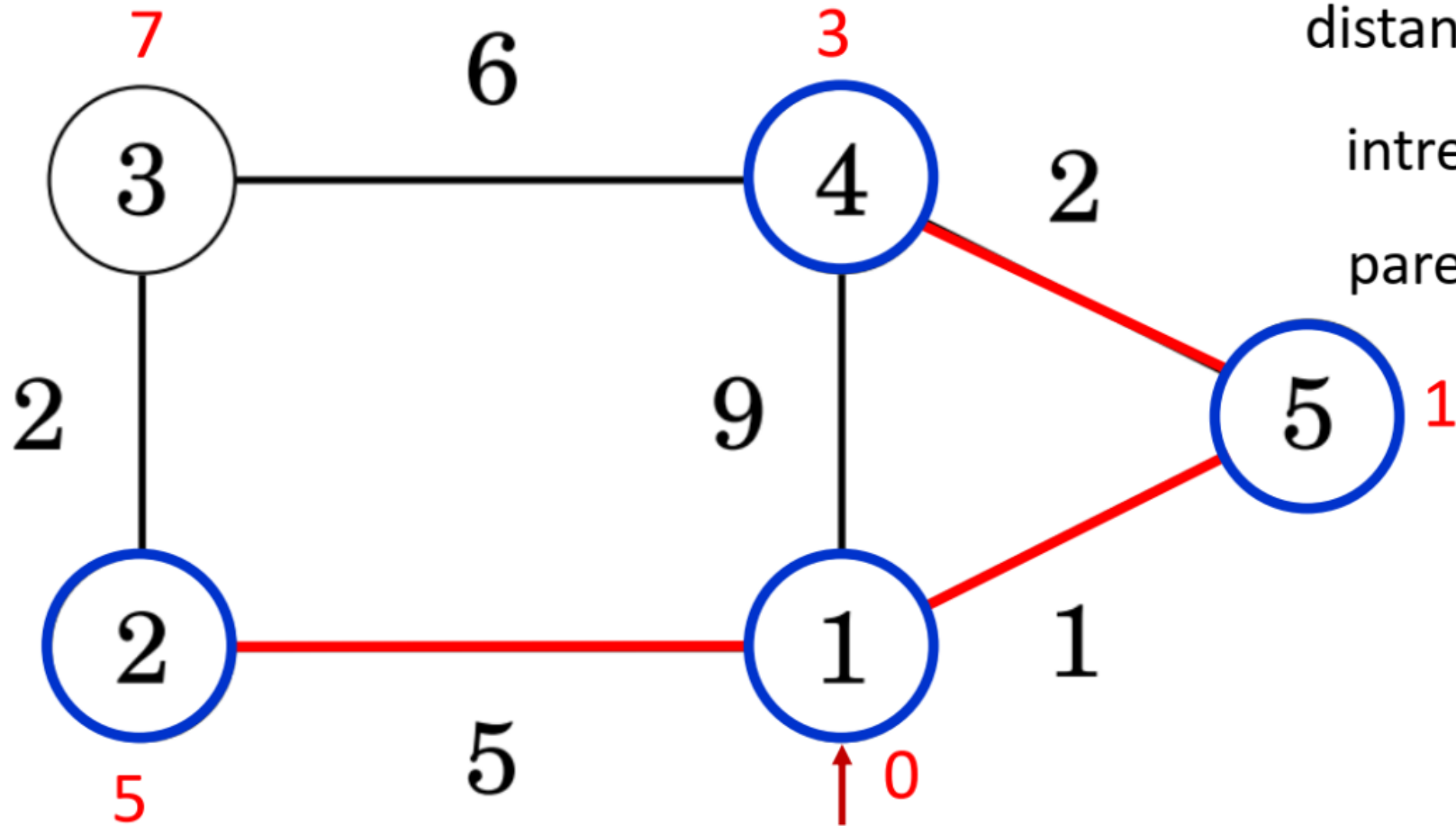
Dijkstra algorithm



	1	2	3	4	5
distance	0	5	9	3	1
intree	1	0	0	1	1
parent	-1	-1	-1	5	1

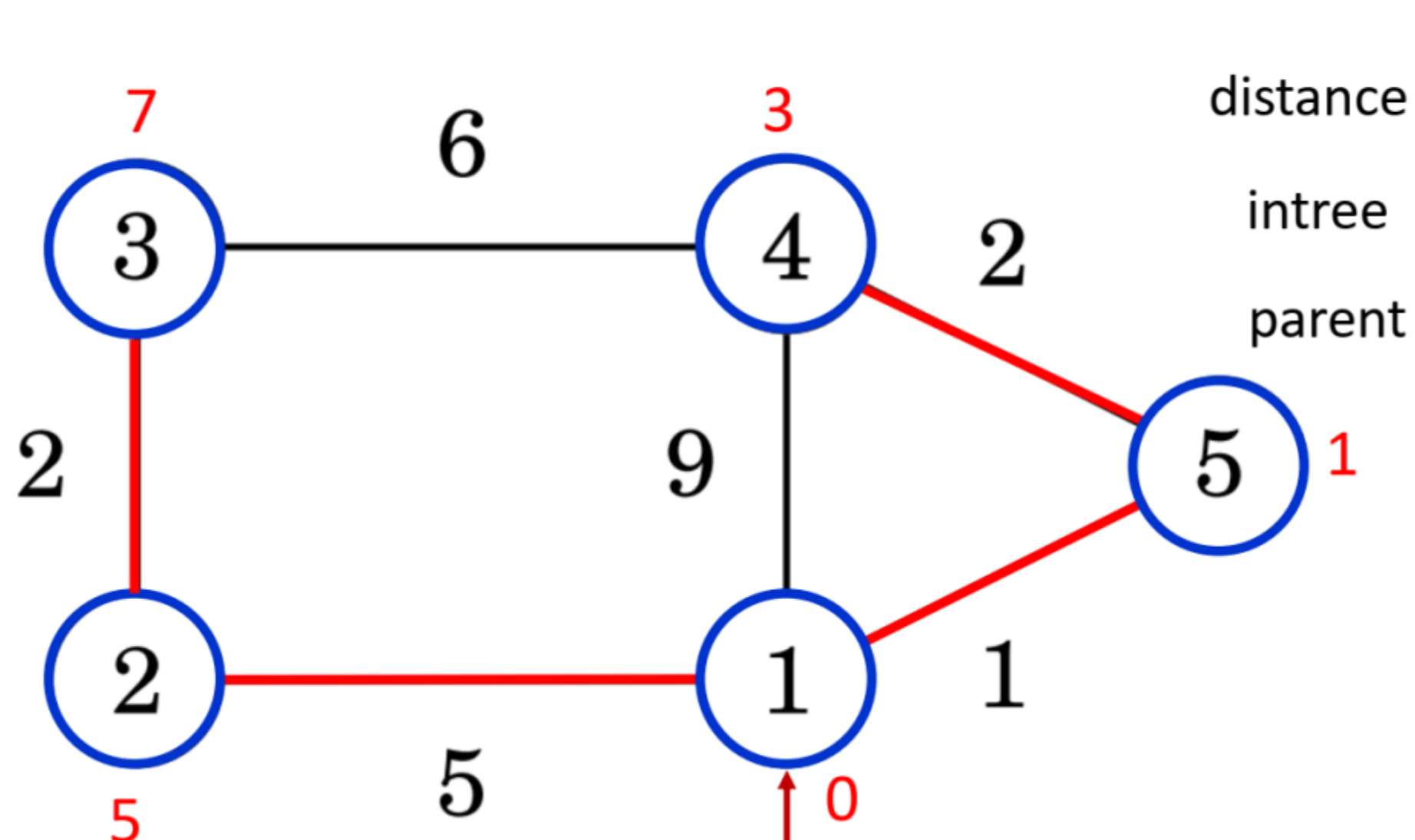


Dijkstra algorithm

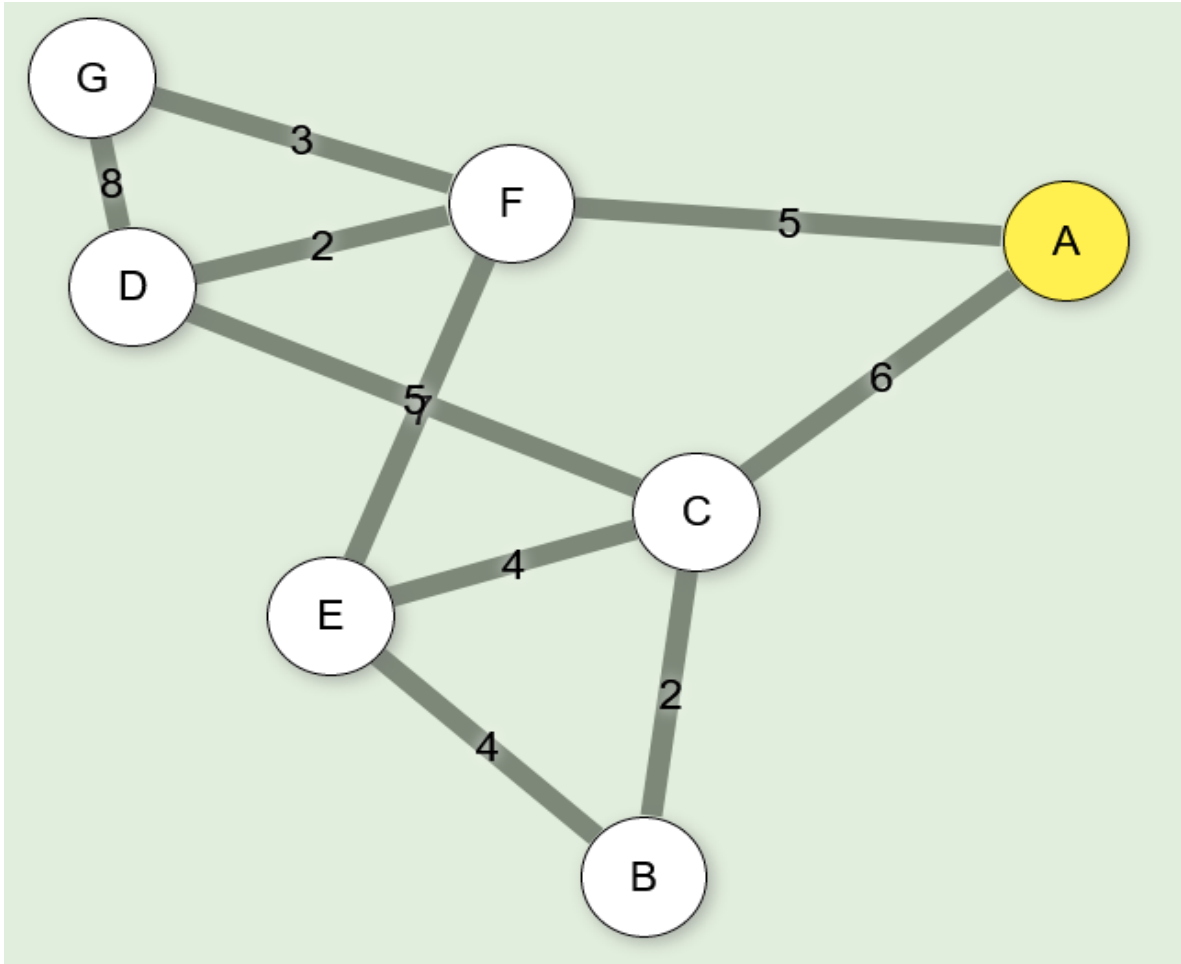


	1	2	3	4	5
distance	0	5	7	3	1
intree	1	1	0	1	1
parent	-1	1	-1	5	1

Dijkstra algorithm



	1	2	3	4	5
distance	0	5	7	3	1
intree	1	1	1	1	1
parent	-1	1	2	5	1



จงแสดงขั้นตอนวิธีการหาเส้นทางที่สั้นที่สุดจาก **A** ไปยัง
เวอร์เทกซ์อื่นในกราฟ โดยใช้อัลกอริทึม **Dijkstra**

แสดงผลลัพธ์ในรูปแบบของอาร์เรย์ **Distance** และอาร์เรย์
Parent-child

Dijkstra algorithm implemented by adj List

```
#include <iostream>
#include <vector>
#include <queue>
```

```
int main()
{   int n = 5;
```

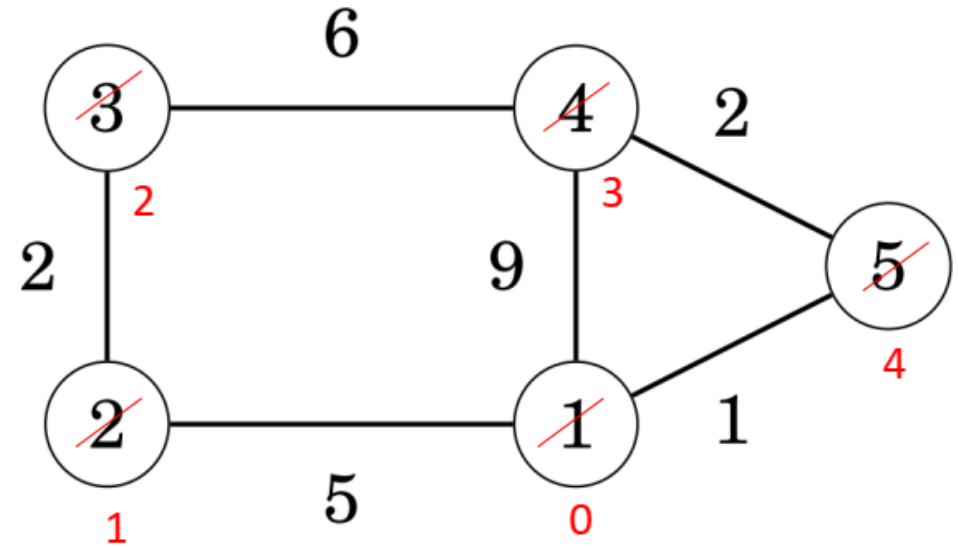
```
    vector<pair<int, int > > adj[n];
```

```
    adj[0].push_back({1, 5});   adj[1].push_back({0, 5});
    adj[0].push_back({3, 9});   adj[3].push_back({0, 9});
    adj[0].push_back({4, 1});   adj[4].push_back({0, 1});
```

```
    adj[1].push_back({2, 2});   adj[2].push_back({1, 2});
    adj[2].push_back({3, 6});   adj[3].push_back({2, 6});
    adj[3].push_back({4, 2});   adj[4].push_back({3, 2});
```

```
    Dijkstra(adj, n, 0);        // start at vertex 0
```

```
}
```



```

void Dijkstra(vector<pair<int,int>> adj[], int V, int src)
{
    priority_queue< pair<int, int>, vector <pair<int, int>> , greater<pair<int, int>> > pq;

    vector<int> dist(V, INF);
    vector<int> parent(V, -1);
    vector<bool> intree(V, false);

    pq.push( {0, src} );           // push start vertex src { dist, vertex}
    dist[src] = 0;                 // initial distance of src

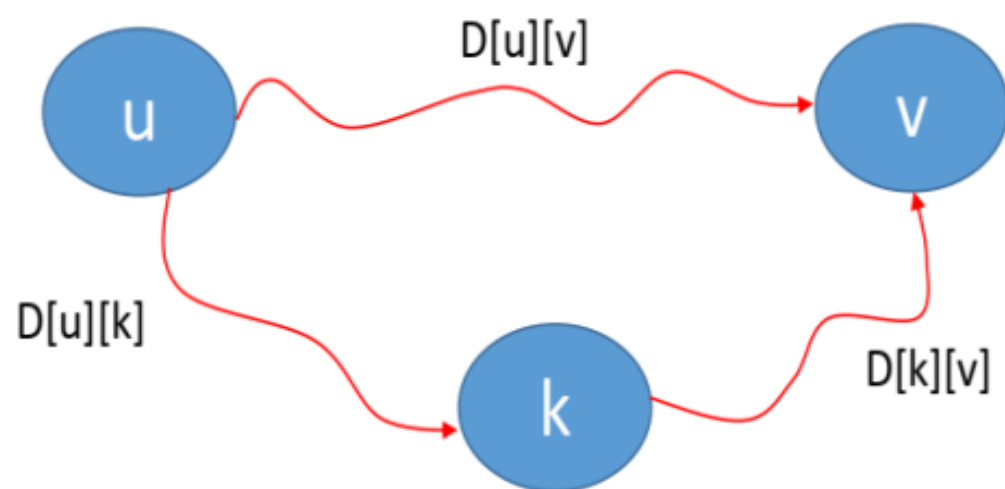
    while (!pq.empty())
    {
        int u = pq.top().second;   // get u from pq.
        intree[u] = true;
        pq.pop();
        for (int j = 0; j < adj[u].size(); ++j) {
            int v = adj[u][j].first; // get v and weight from adj list.
            int weight = adj[u][j].second;

            if (intree[v] == false && dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight; // Update weight of v
                pq.push({dist[v], v});      // push v into pq.
                parent[v] = u;
            }
        }
    }
}

```

อัลกอริทึม Floyd-Warshall

แนวคิดอัลกอริทึม Floyd คือการสร้างเมตริกซ์ระยะทาง **D** (distance matrix) เพื่อใช้เก็บระยะทางที่สั้นที่สุดจาก u ไป v โดยที่ u และ v เป็นเวอร์เท็กซ์ในกราฟ จากนั้นจะมีการปรับปรุงค่าระยะทางในทุกๆ คู่ หากพบว่ามีความสั้นกว่าระยะทางเดิมที่เคยบันทึกไว้ก่อนหน้านี้



แนวคิดของ Floyd-Warshall

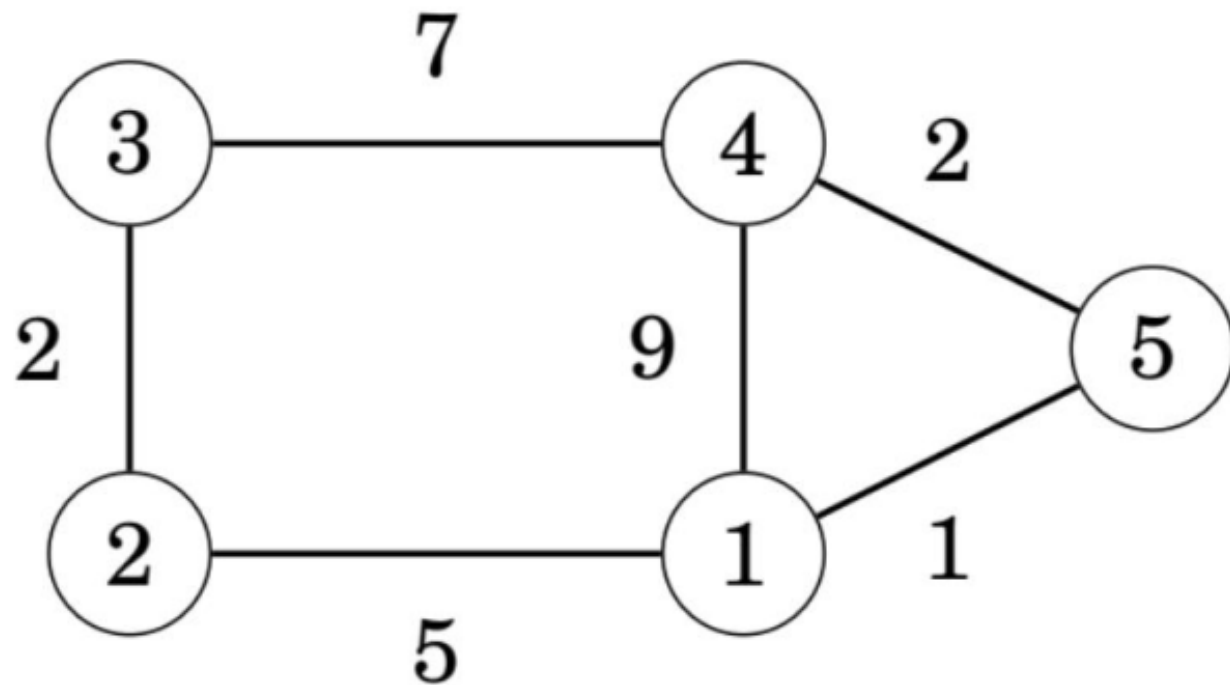
$$D[u][v] = \min \{ D[u][v], D[u][k] + D[k][v] \}$$

โดยที่ k เป็น intermediate vertex

k = เริ่มต้นจากเวอร์เท็กซ์ 0 ไปยังเวอร์เท็กซ์ $n-1$

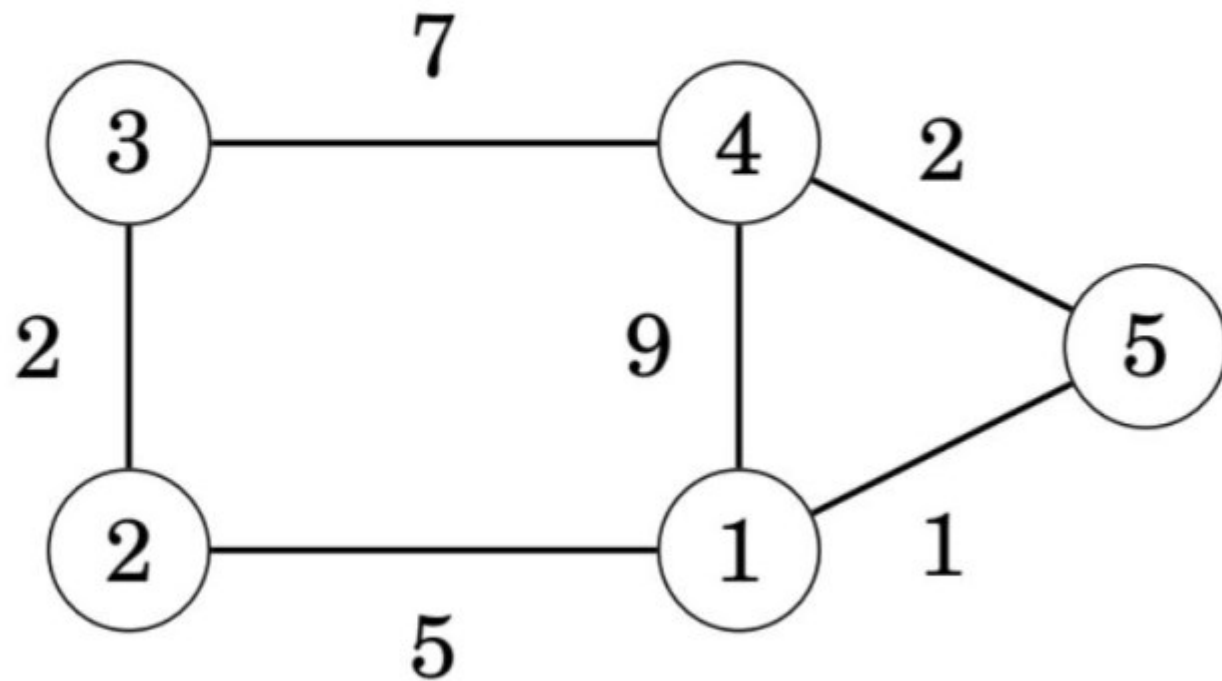
ตัวอย่าง

- เตรียมเมตริกซ์ D โดยเริ่มจากค่าระยะทางในเอดจ์ในกราฟ
- สำหรับ $D[i][i] = 0$ และหากไม่มีเอดจ์ในคู่เวอร์เท็กซ์ใดๆ จะกำหนดมีค่าเท่ากับ ∞



	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	∞	∞
3	∞	2	0	7	∞
4	9	∞	7	0	2
5	1	∞	∞	2	0

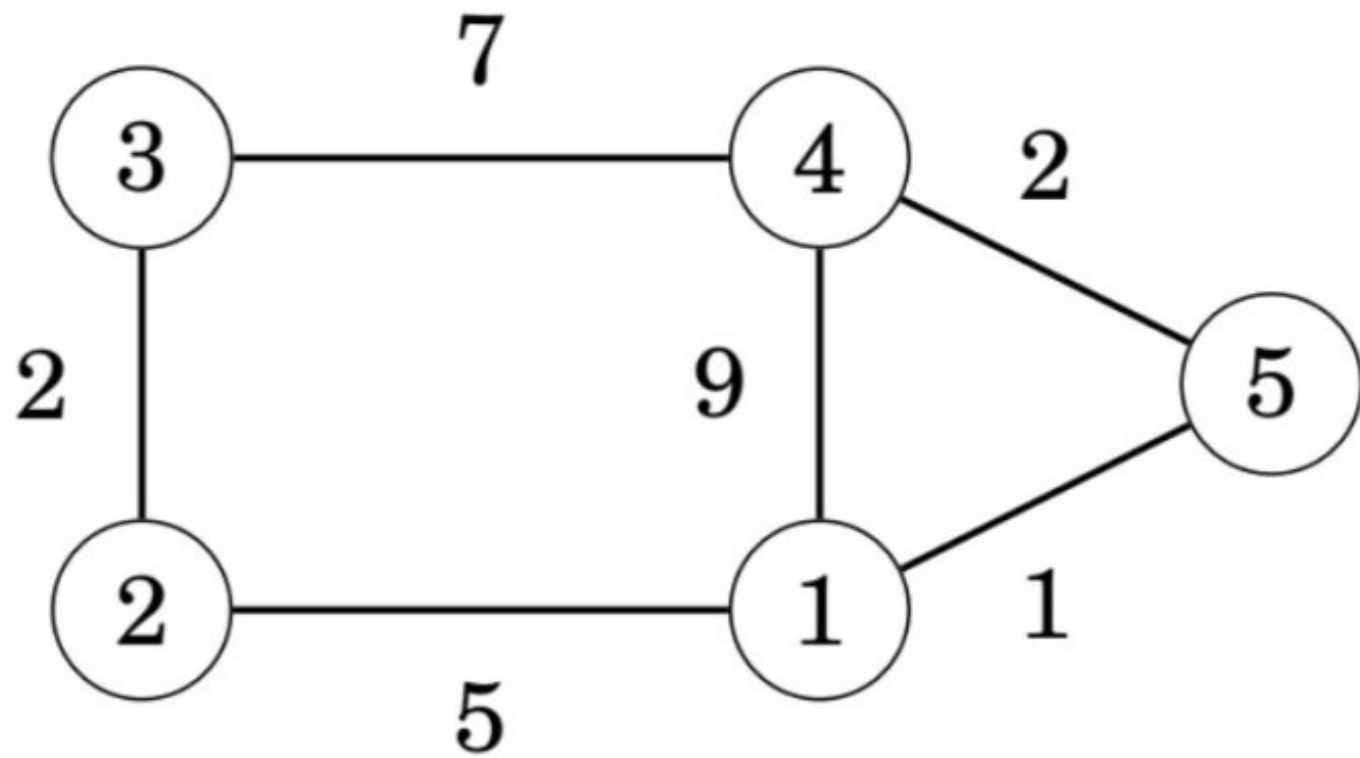
เวอร์เท็กซ์ $k = 1$



	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	∞	∞
3	∞	2	0	7	∞
4	9	∞	7	0	2
5	1	∞	∞	2	0

	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	14	6
3	∞	2	0	7	∞
4	9	14	7	0	2
5	1	6	∞	2	0

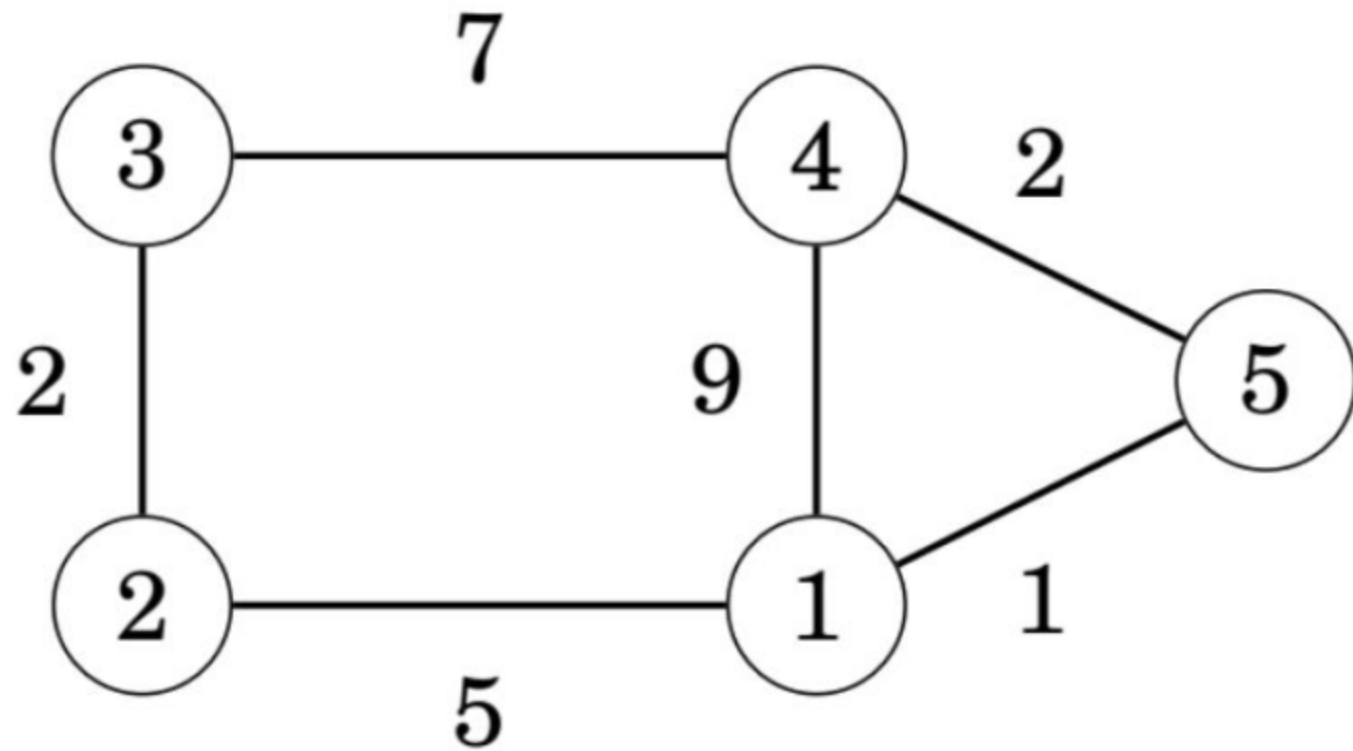
เวอร์เท็กซ์ $k = 2$



	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	14	6
3	∞	2	0	7	∞
4	9	14	7	0	2
5	1	6	∞	2	0

	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	14	6
3	7	2	0	7	8
4	9	14	7	0	2
5	1	6	8	2	0

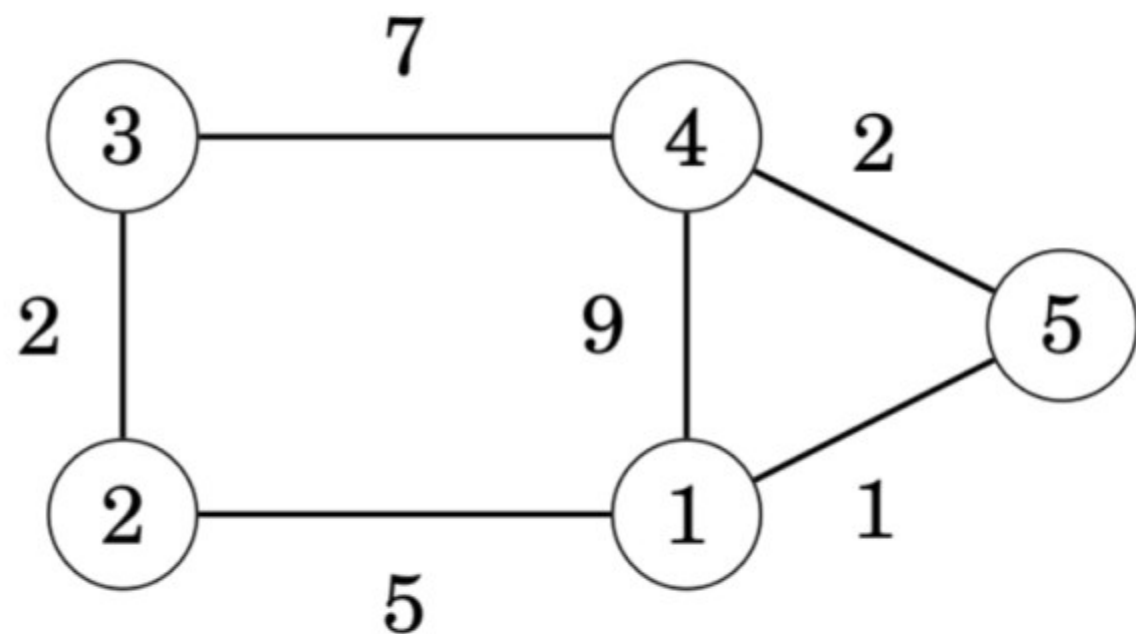
เวอร์เท็กซ์ $k = 3$



	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	14	6
3	7	2	0	7	8
4	9	14	7	0	2
5	1	6	8	2	0

	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	9	6
3	7	2	0	7	8
4	9	9	7	0	2
5	1	6	8	2	0

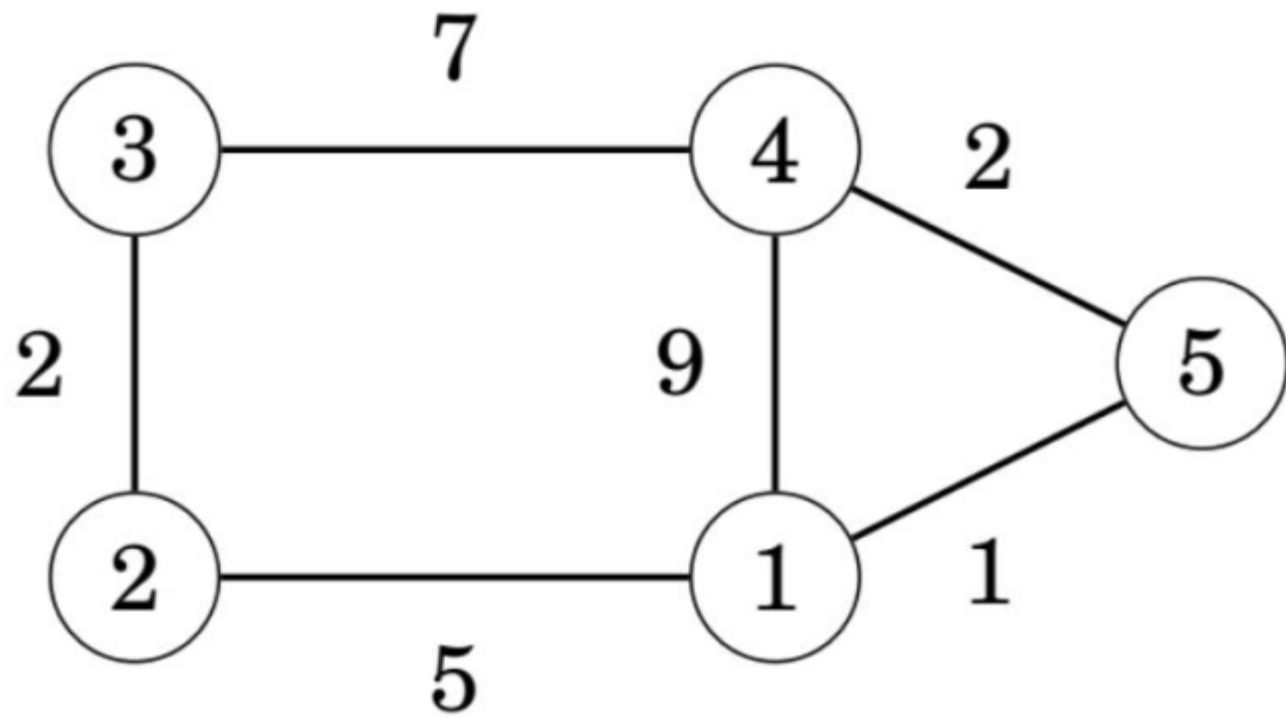
เวอร์เท็กซ์ $k = 4$



	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	9	6
3	7	2	0	7	8
4	9	9	7	0	2
5	1	6	8	2	0

	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	9	6
3	7	2	0	7	8
4	9	9	7	0	2
5	1	6	8	2	0

เวอร์เท็กซ์ $k = 5$



	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	9	6
3	7	2	0	7	8
4	9	9	7	0	2
5	1	6	8	2	0

	1	2	3	4	5
1	0	5	7	3	1
2	5	0	2	8	6
3	7	2	0	7	8
4	3	8	7	0	2
5	1	6	8	2	0

```
void floydWarshall(int graph[][V]) {
```

```
    int dist[V][V];
```

```
    for (int i = 0; i < V; i++)
```

```
        for (int j = 0; j < V; j++)
```

```
            dist[i][j] = graph[i][j];    // copy adj matrix -> dist
```

$O(n^3)$

```
    for (int k = 0; k < V; k++) {
```

```
        for (int i = 0; i < V; i++) {
```

```
            for (int j = 0; j < V; j++) {
```

```
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
```

```
                    dist[i][j] = dist[i][k] + dist[k][j];
```

```
                }
```

```
            }
```

```
        }
```

```
    }
```

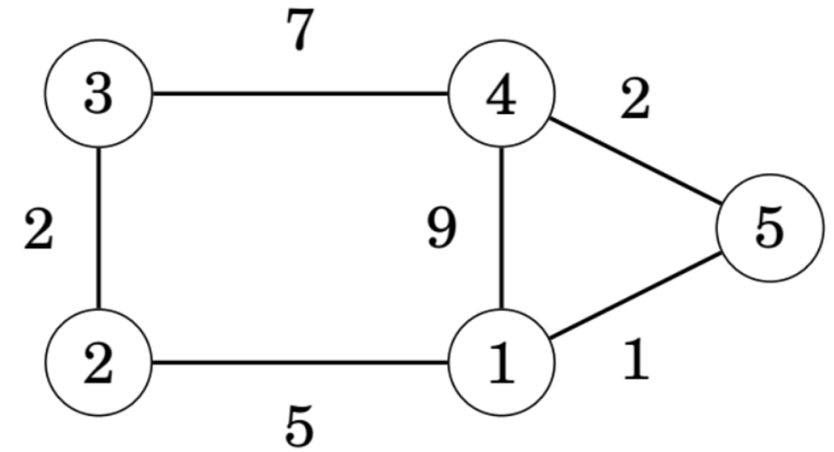
การเก็บ path คำตอบ

สร้างเมตริกซ์ path เพิ่ม เพื่อเก็บ vertex ถัดไปที่จะต้องผ่านเมื่อเดินจาก u ไป v

```
for(i=0; i<V; i++)
{
    for(j=0; j<V; j++)
    {
        if(graph[i][j] != INF)
            path[i][j] = j;
        else
            path[i][j] = -1;
    }
}
```

ขณะปรับปรุง dist[i][j] หากพบว่าผ่าน k แล้วเส้นทางสั้นกว่าเดิม ก็จะมีการปรับปรุง path ตามไปด้วย

```
if(dist[i][k] + dist[k][j] < dist[i][j]) {
    dist[i][j] = dist[i][k] + dist[k][j];
    path[i][j] = path[i][k];
}
```



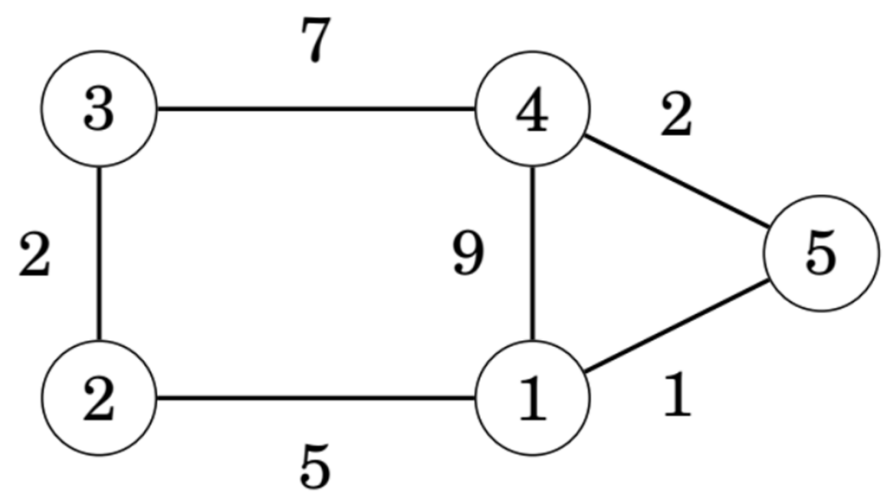
	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	∞	∞
3	∞	2	0	7	∞
4	9	∞	7	0	2
5	1	∞	∞	2	0

Dist

	1	2	3	4	5
1	1	2	-1	4	5
2	1	2	3	-1	-1
3	-1	2	3	4	-1
4	1	-1	3	4	5
5	5	-1	-1	4	5

Path

ตัวอย่าง



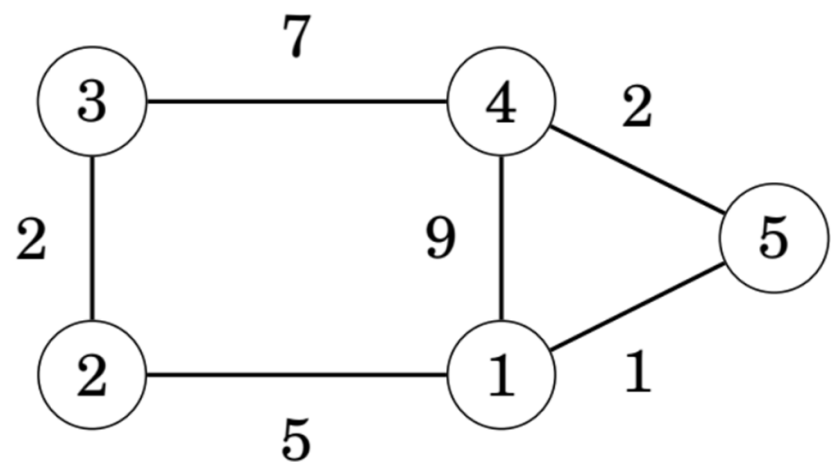
path

	1	2	3	4	5
1	1	2	-1	4	5
2	1	2	3	-1	-1
3	-1	2	3	4	-1
4	1	-1	3	4	5
5	5	-1	-1	4	5

	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	∞	∞
3	∞	2	0	7	∞
4	9	∞	7	0	2
5	1	∞	∞	2	0

graph

ตัวอย่าง $k = 1$



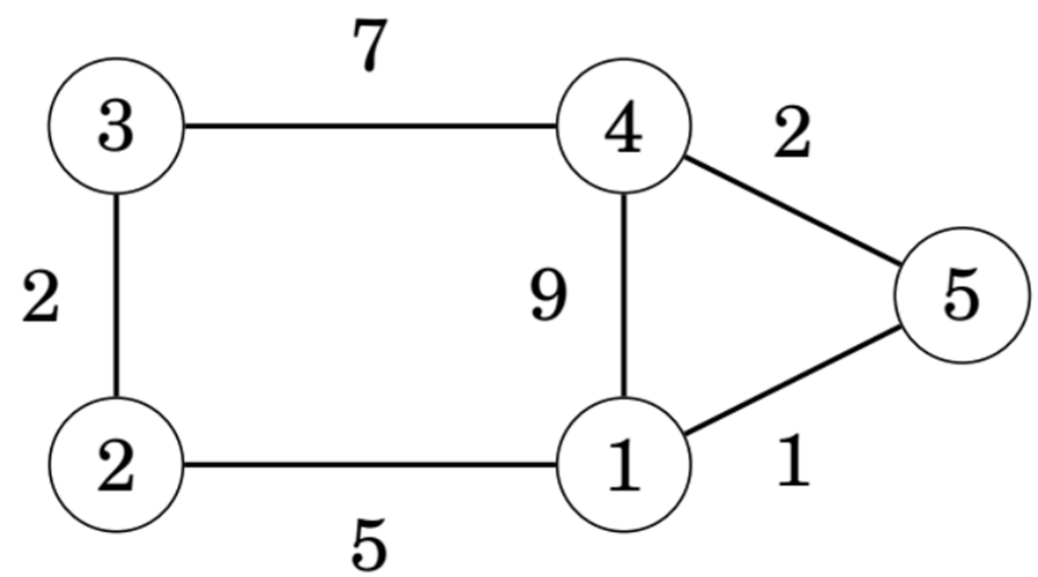
path

	1	2	3	4	5
1	1	2	-1	4	5
2	1	2	3	1	1
3	-1	2	3	4	-1
4	1	1	3	4	5
5	5	1	-1	4	5

	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	∞	∞
3	∞	2	0	7	∞
4	9	∞	7	0	2
5	1	∞	∞	2	0

	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	14	6
3	∞	2	0	7	∞
4	9	14	7	0	2
5	1	6	∞	2	0

ตัวอย่าง $k = 2$



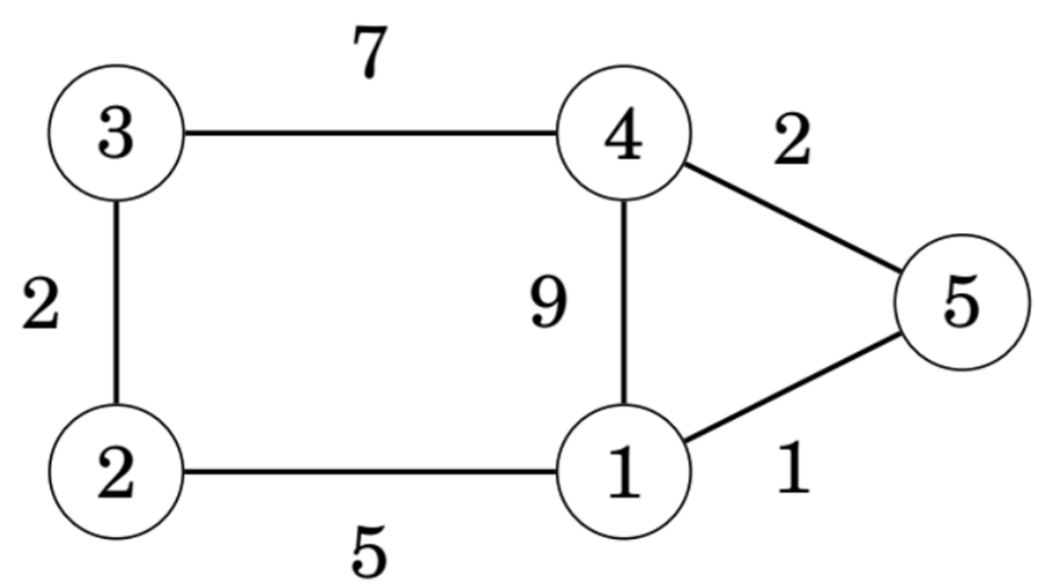
path

	1	2	3	4	5
1	1	2	2	4	5
2	1	2	3	1	1
3	2	2	3	4	2
4	1	1	3	4	5
5	5	1	2	4	5

	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	14	6
3	∞	2	0	7	∞
4	9	14	7	0	2
5	1	6	∞	2	0

	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	14	6
3	7	2	0	7	8
4	9	14	7	0	2
5	1	6	8	2	0

ตัวอย่าง k = 3



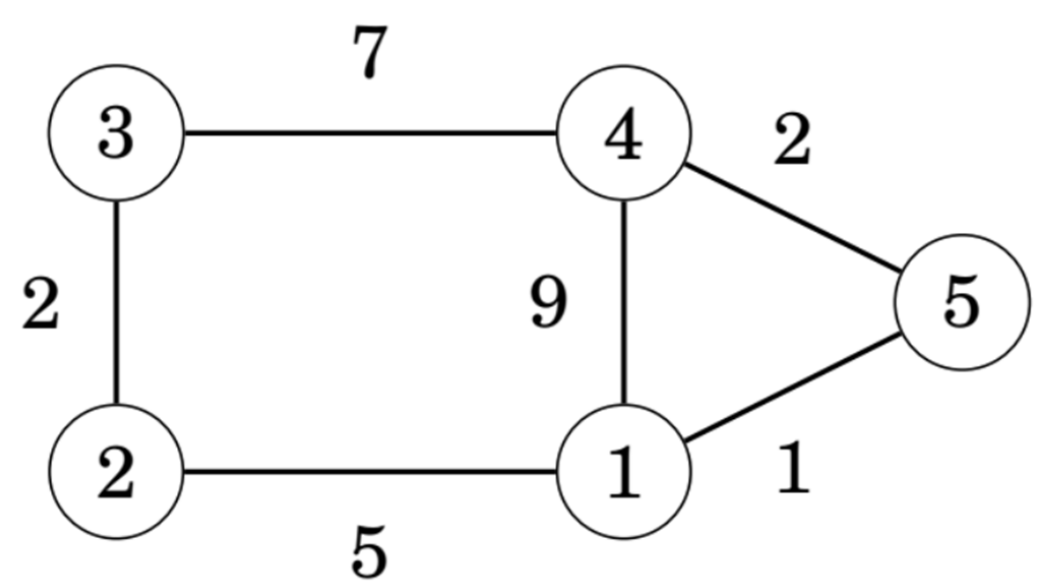
path

	1	2	3	4	5
1	1	2	2	4	5
2	1	2	3	3	1
3	2	2	3	4	2
4	1	3	3	4	5
5	5	1	2	4	5

	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	14	6
3	7	2	0	7	8
4	9	14	7	0	2
5	1	6	8	2	0

	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	9	6
3	7	2	0	7	8
4	9	9	7	0	2
5	1	6	8	2	0

ตัวอย่าง k = 4



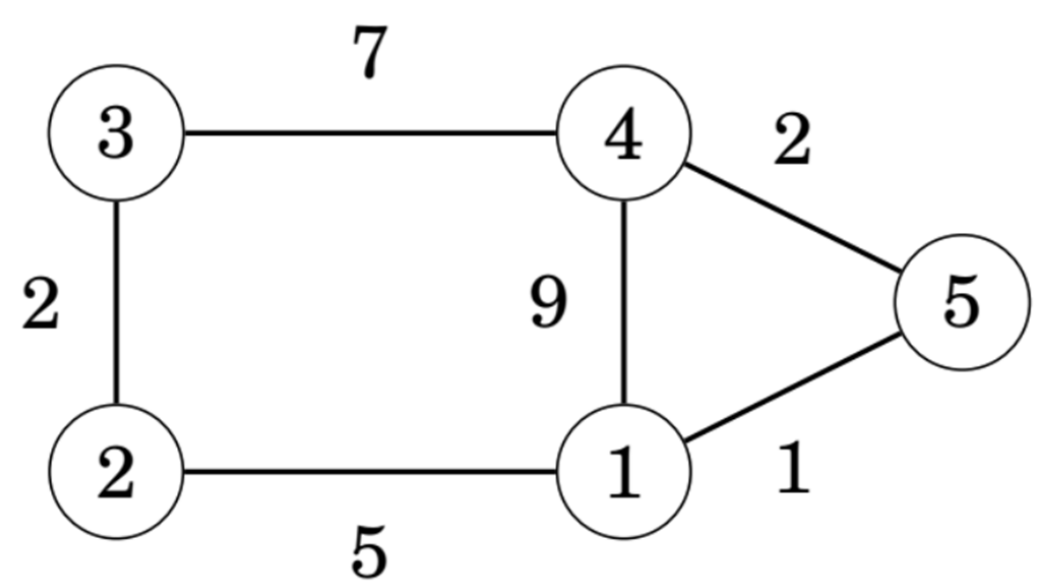
path

	1	2	3	4	5
1	1	2	2	4	5
2	1	2	3	3	1
3	2	2	3	4	2
4	1	3	3	4	5
5	5	1	2	4	5

	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	9	6
3	7	2	0	7	8
4	9	9	7	0	2
5	1	6	8	2	0

	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	9	6
3	7	2	0	7	8
4	9	9	7	0	2
5	1	6	8	2	0

ตัวอย่าง k = 5



path

	1	2	3	4	5
1	1	2	2	5	5
2	1	2	3	5	1
3	2	2	3	4	2
4	5	5	3	4	5
5	5	1	2	4	5

	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	9	6
3	7	2	0	7	8
4	9	9	7	0	2
5	1	6	8	2	0

	1	2	3	4	5
1	0	5	7	3	1
2	5	0	2	8	6
3	7	2	0	7	8
4	3	8	7	0	2
5	1	6	8	2	0

Path reconstruction

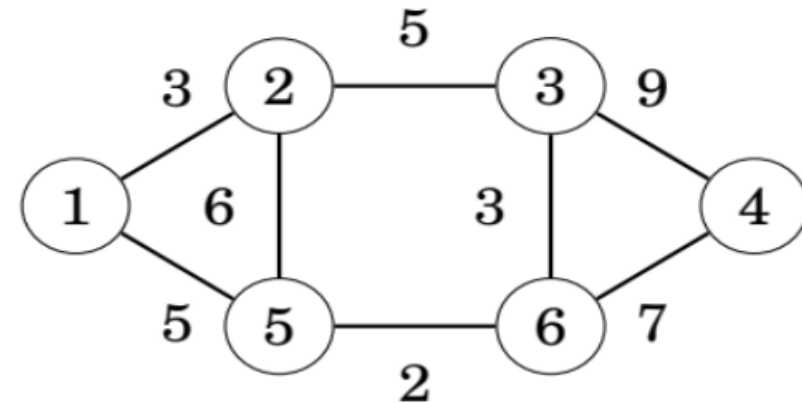
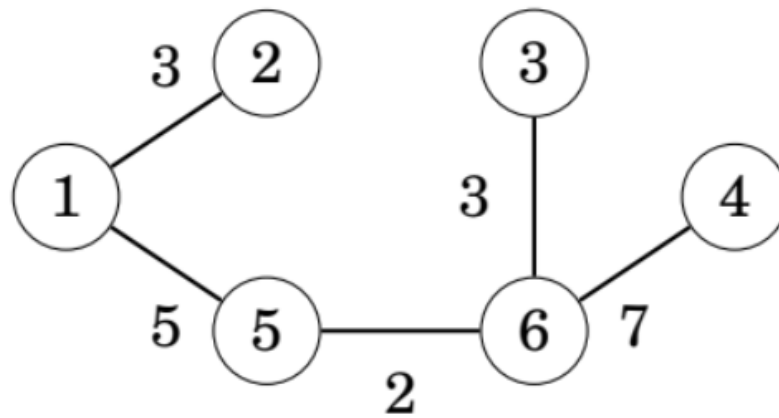
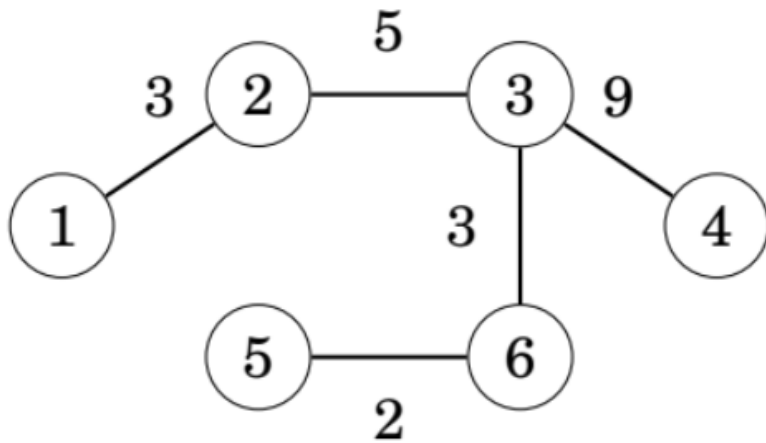
```
void printPath(int u, int v, int path[V][V])
{ if(path[u][v] == -1)
  { printf("No path\n");
    return;
  }
  printf("Path: %d", u);
  while(u != v)
  { u = path[u][v];
    printf(" -> %d", u);
  }
  printf("\n");
}
```

	1	2	3	4	5
1	1	2	2	5	5
2	1	2	3	5	1
3	2	2	3	4	2
4	5	5	3	4	5
5	5	1	2	4	5

```
printPath(1, 4, path)
1 -> 5 -> 4
```

ต้นไม้แผ่ทั่วเล็กที่สุด (Minimum Spanning Tree)

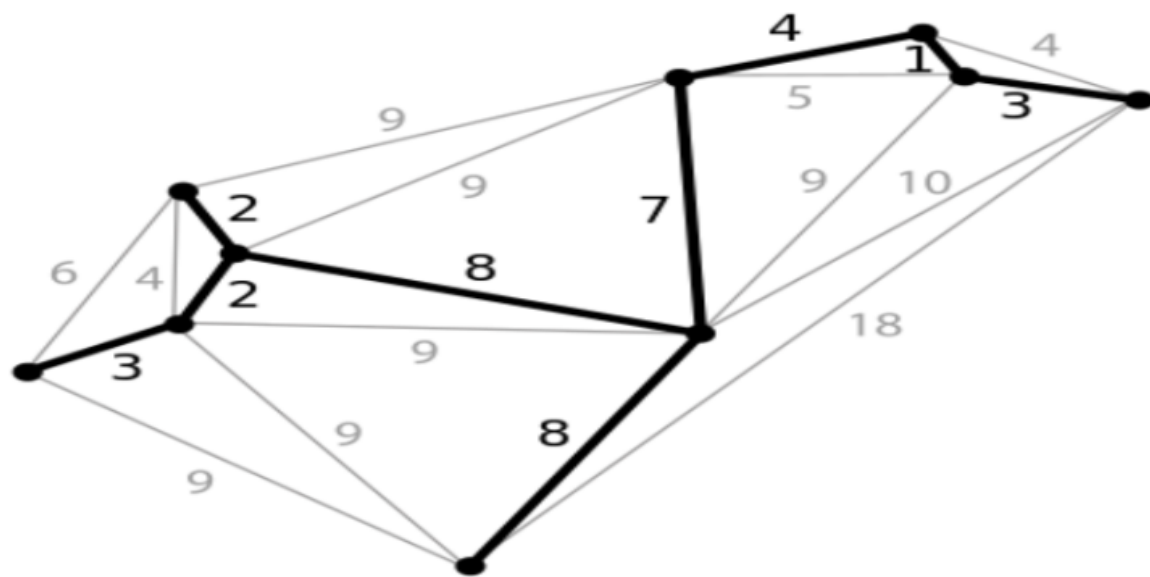
- ต้นไม้แผ่ทั่ว (Spanning tree) หมายถึง acyclic graph ซึ่งเชื่อมโยงทุกเวอร์เท็กซ์ด้วยเอจด์บางเส้น ($|V| - 1$)
- ต้นไม้แผ่ทั่วเล็กที่สุด (Minimum spanning tree) คือต้นไม้แผ่ทั่วที่ผลรวมของเอจด์ที่น้อยที่สุด





ต้นไม้แผ่ทั่วเล็กที่สุด (Minimum Spanning Tree)

- MST ในกราฟอาจมีได้มากกว่า 1 คำตอบ
- การสำรวจกราฟด้วย BFS /DFS เพื่อหา MST โดยตรงอาจไม่เหมาะสม เนื่องจากจำนวนต้นไม้แผ่ทั่วในกราฟมีมากเกินไปที่จะสร้างออกมาได้ทั้งหมด (exponential time)
- อัลกอริทึมสำหรับการค้นหาต้นไม้แผ่ทั่วเล็กที่สุด
 - อัลกอริทึม Prim
 - อัลกอริทึม Kruskal

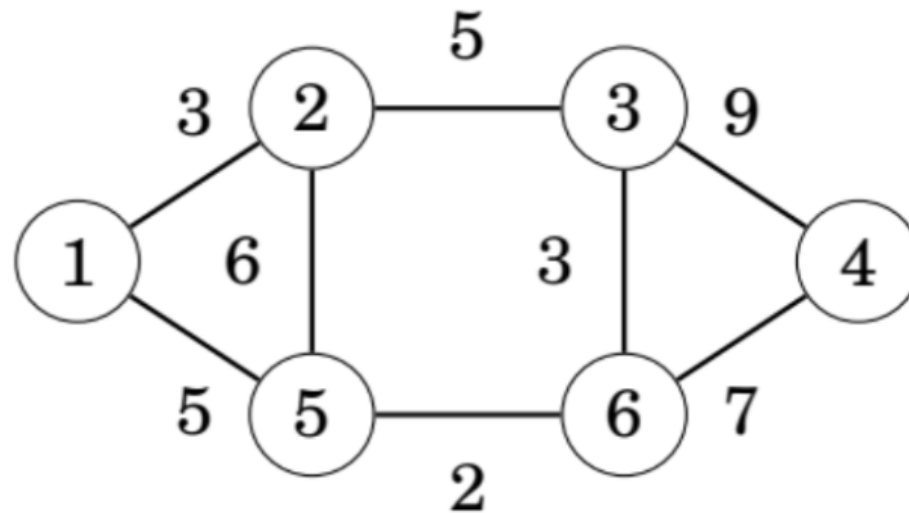




อัลกอริทึม Prim

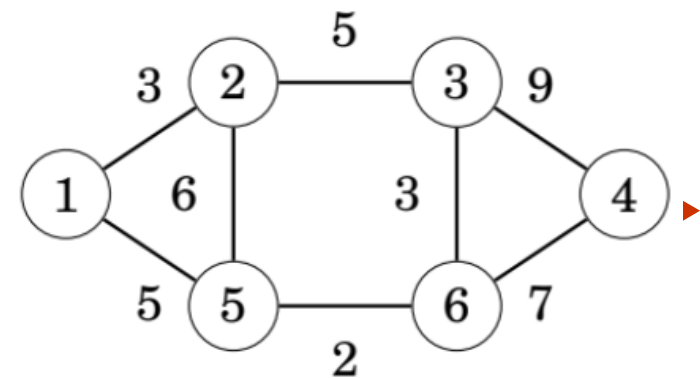
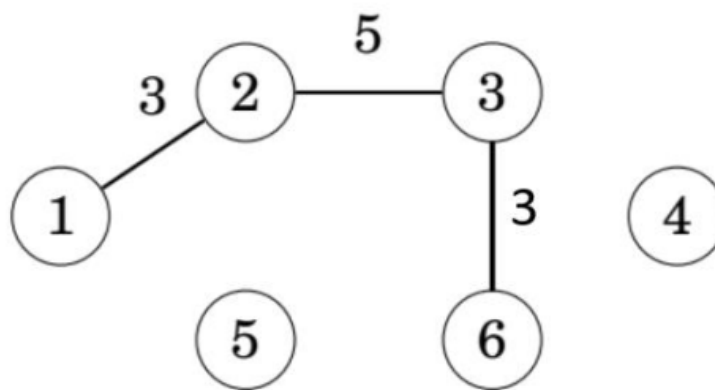
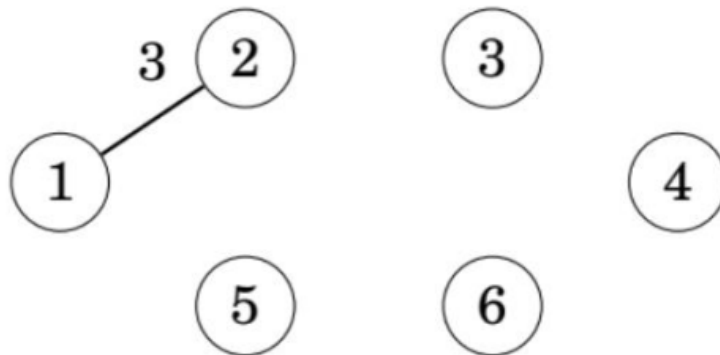
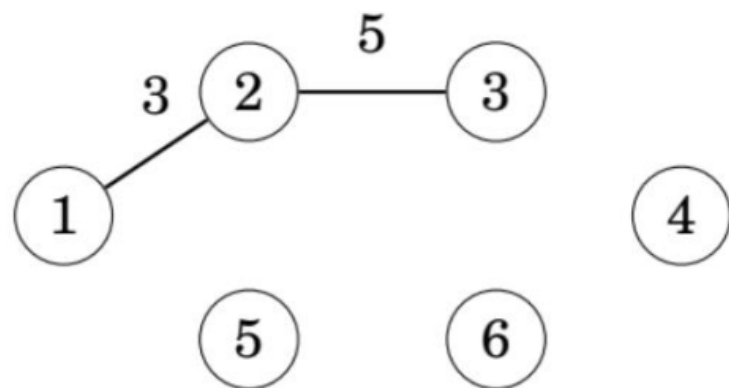
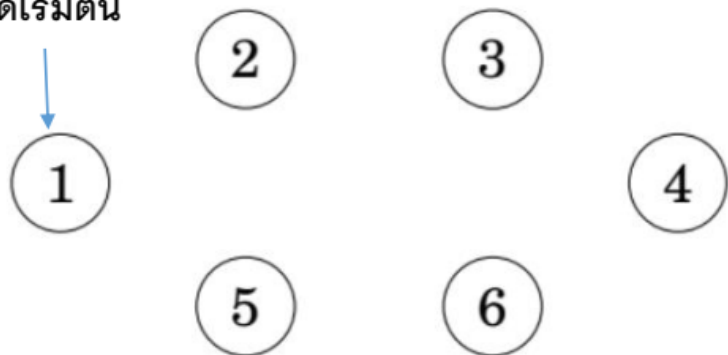


- เป็นอัลกอริทึมค้นหาต้นไม้แผ่ทั่วเล็กที่สุดที่คล้ายคลึงกับอัลกอริทึม Dijkstra
- โดยเริ่มต้นจะต้องกำหนดเวอร์เท็กซ์เริ่มต้นในกราฟ จากนั้นอัลกอริทึมจะเลือกเอจด์ที่มีค่าน้อยที่สุด (minimum-weight edge) จากเวอร์เท็กซ์เริ่มต้นและแผ่ทั่วจนครอบคลุมทุกเวอร์เท็กซ์
- อัลกอริทึม Prim จะเลือกเอจด์จนครบทุกเวอร์เท็กซ์ $(|V| - 1)$ โดยที่ V หมายถึงจำนวนเวอร์เท็กซ์



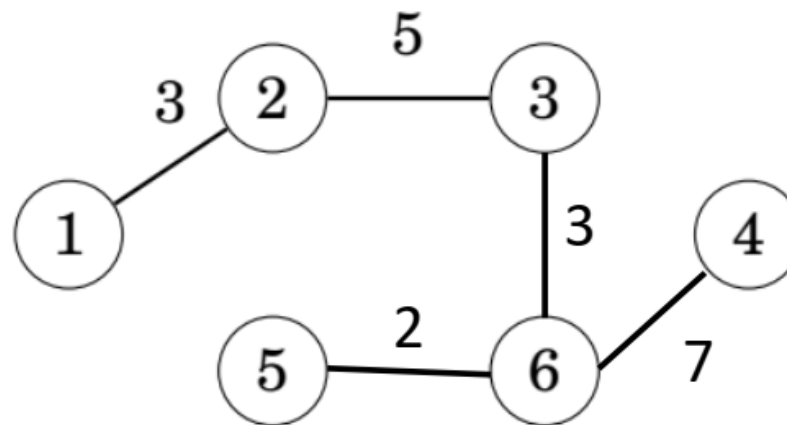
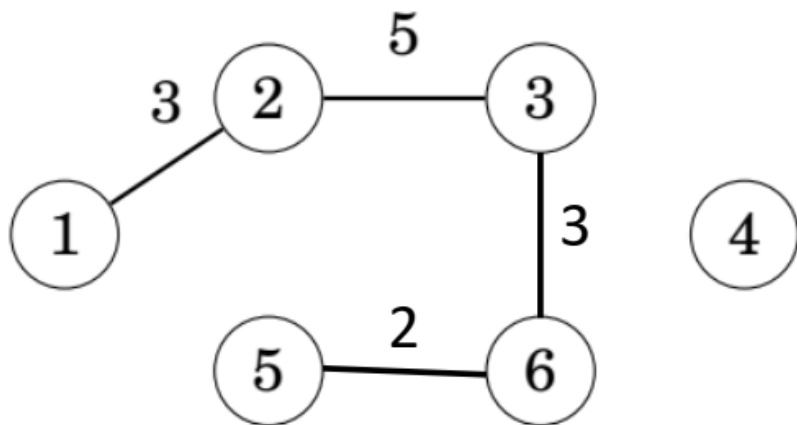
อัลกอริทึม Prim

จุดเริ่มต้น

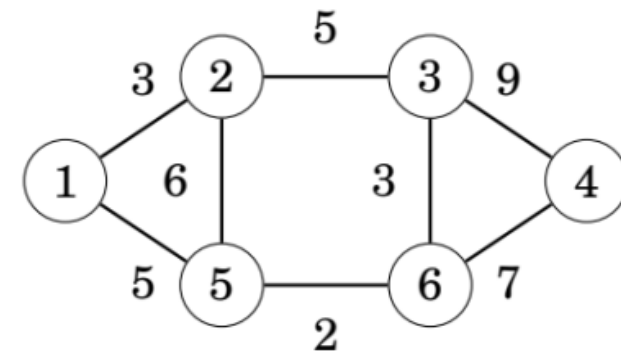


เลือกเอจที่อยูใกล้กับต้นไม่มากที่สุด
จนกระทั่งครบ $V-1$ เอจด์

อัลกอริทึม Prim

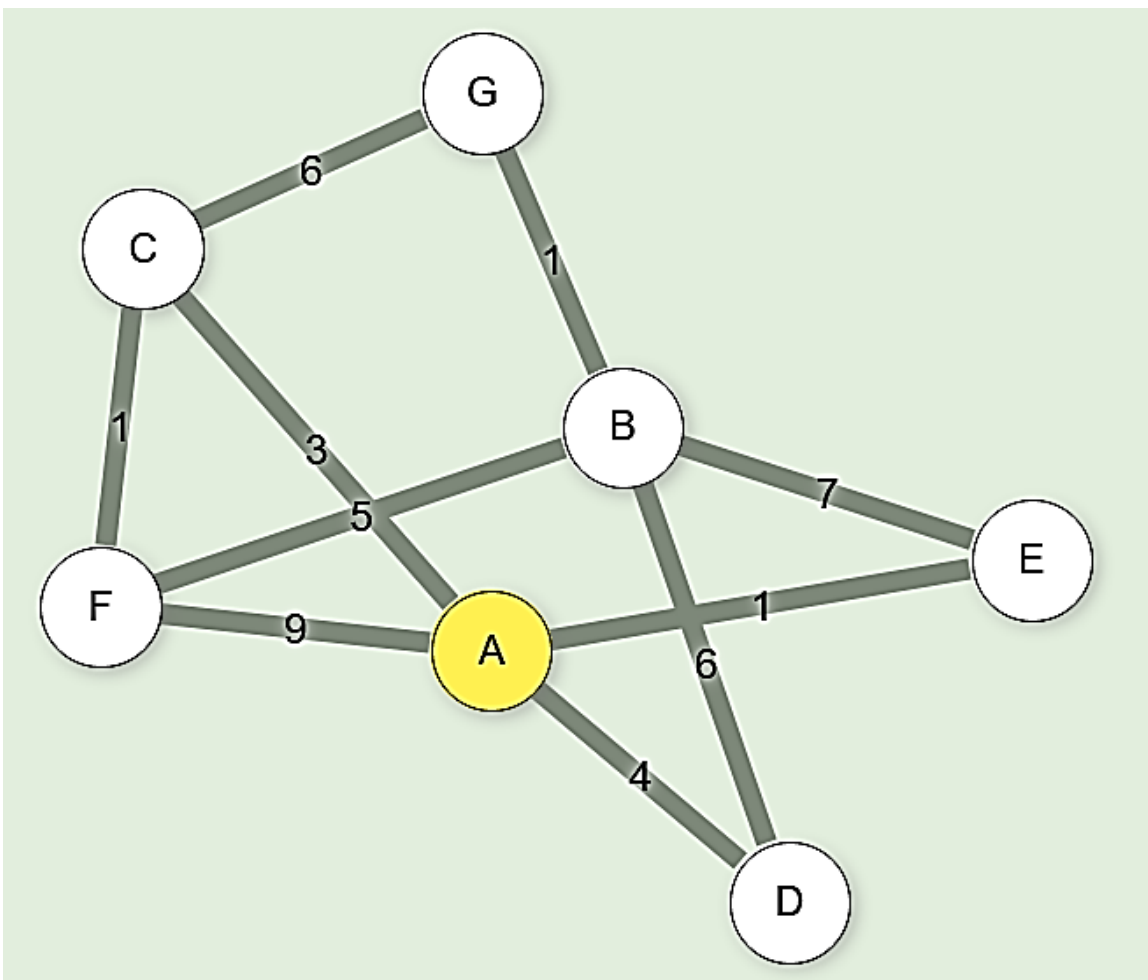


MST
 $3+5+2+3+7 = 20$



เลือกเอ็ดจ์ที่อยู่ใกล้กับต้นไม้มากที่สุด
จนกระทั่งครบ $V-1$ เอ็ดจ์

แบบฝึกหัด



จงแสดงขั้นตอนการค้นหาต้นไม้แผ่ทั่วที่เล็กที่สุดในกราฟดังต่อไปนี้
ด้วยขั้นตอนวิธี Prim กำหนดให้เริ่มต้นจากเวอร์เท็กซ์ A
แสดงในรูปของ distance และ parent

```

void Prim(vector<pair<int,int>> adj[], int V, int src)
{
    priority_queue< pair<int, int>, vector <pair<int, int>> , greater<pair<int, int>> > pq;

    vector<int> key(V, INF);           // keep distance of vertex
    vector<int> parent(V, -1);         // keep path of MST
    vector<bool> inMST(V, false);      // keep status of vertex

    pq.push({ 0, src } );
    key[src] = 0;

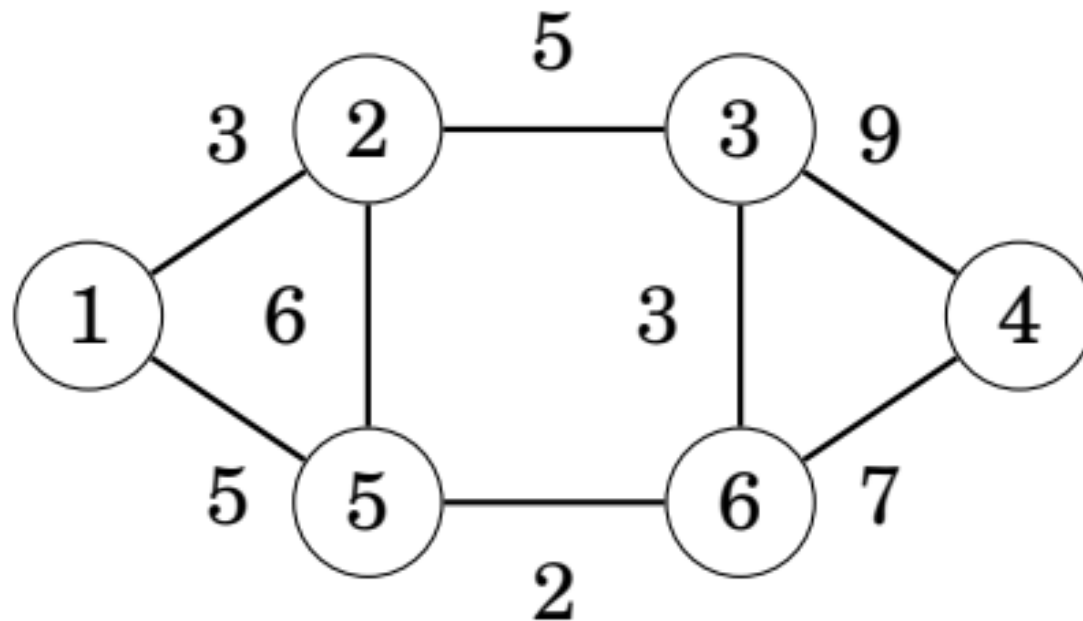
    while (!pq.empty())
    {
        int u = pq.top().second;       // get u from pq.
        pq.pop();
        inMST[u] = true;               // add u into MST

        for (int j = 0; j < adj[u].size(); ++j) {
            int v = adj[u][j].first;   // get v and weight from adj list.
            int weight = adj[u][j].second;
            if (inMST[v] == false && weight < key[v])
            {
                key[v] = weight;
                pq.push({ key[v], v }); // push v into pq.
                parent[v] = u;
            }
        }
    }
}

```

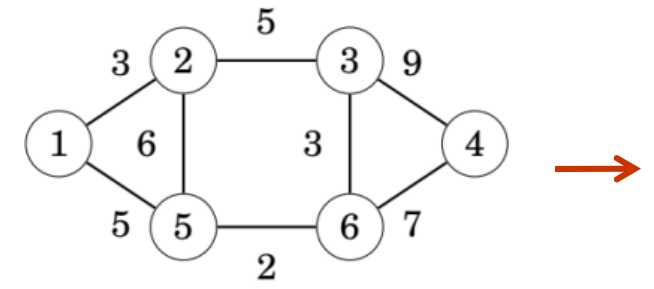
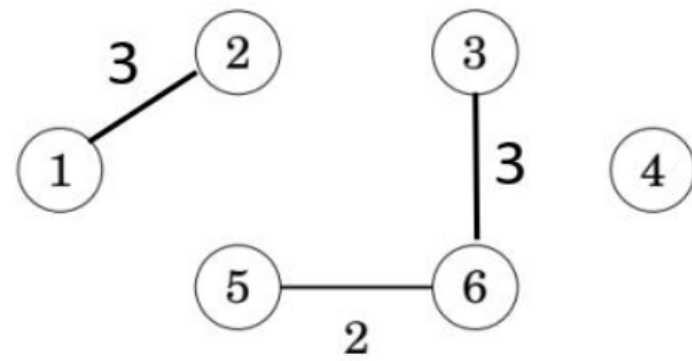
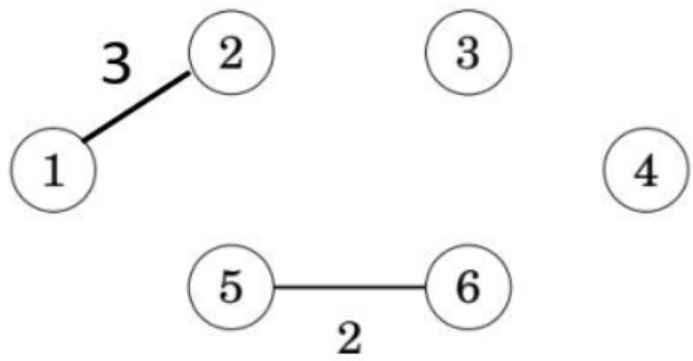
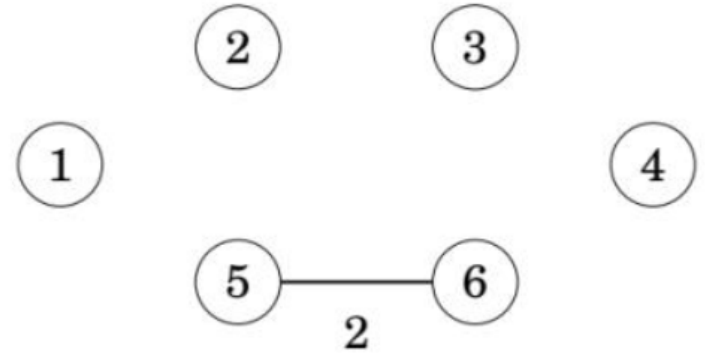
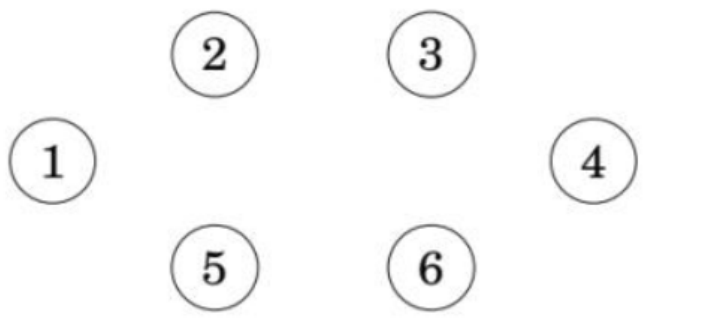
Kruskal's algorithm

- เริ่มต้นจากเรียงลำดับเอดจ์ (edge) ตามค่าน้ำหนักจากน้อยไปมาก
- เพิ่มเอดจ์เข้าไปในคำตอบ หากเอดจ์นั้นไม่ทำให้เกิด วงวน (cycle) จนกระทั่งครอบคลุมทุกเวอร์เท็กซ์





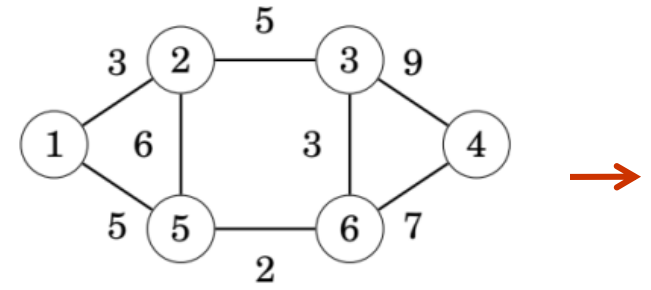
Kruskal's algorithm



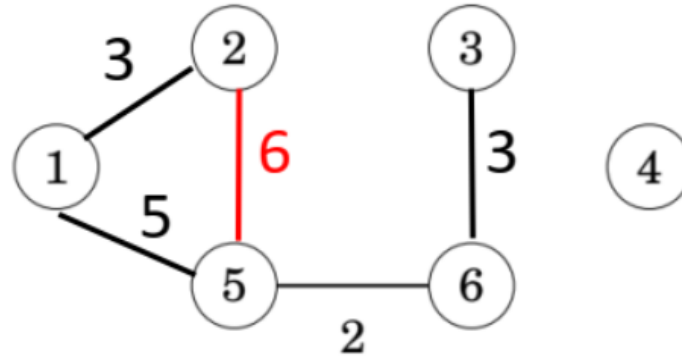
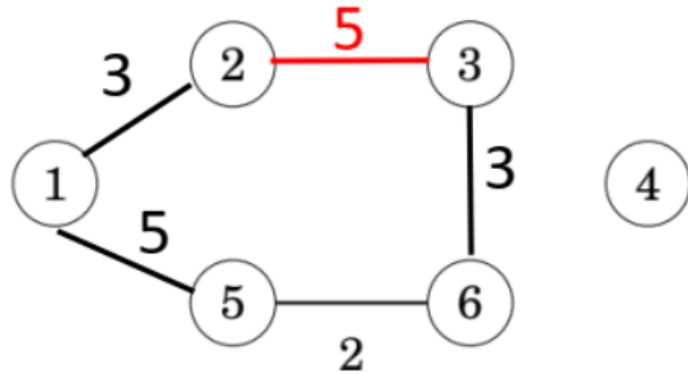
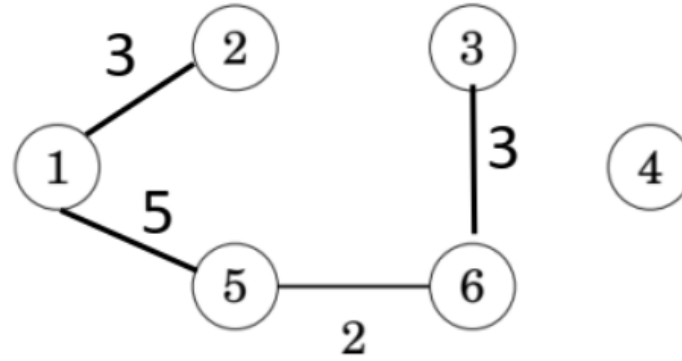
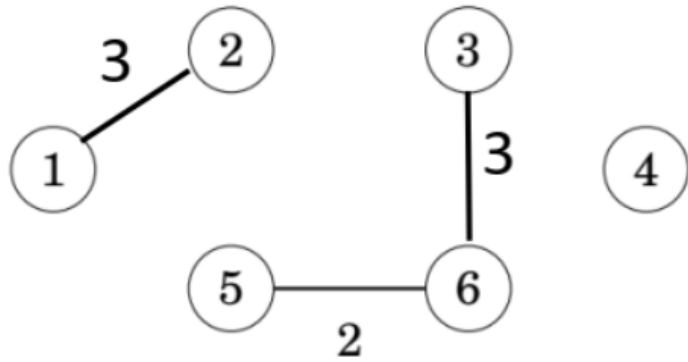
edge	weight
5-6	2
1-2	3
3-6	3
1-5	5
2-3	5
2-5	6
4-6	7
3-4	9



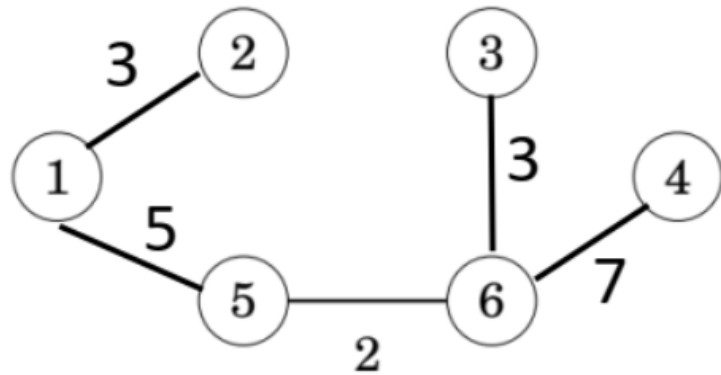
Kruskal's algorithm



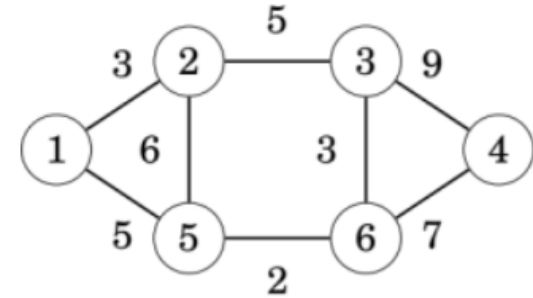
edge	weight
5-6	2
1-2	3
3-6	3
1-5	5
2-3	5
2-5	6
4-6	7
3-4	9



Kruskal's algorithm

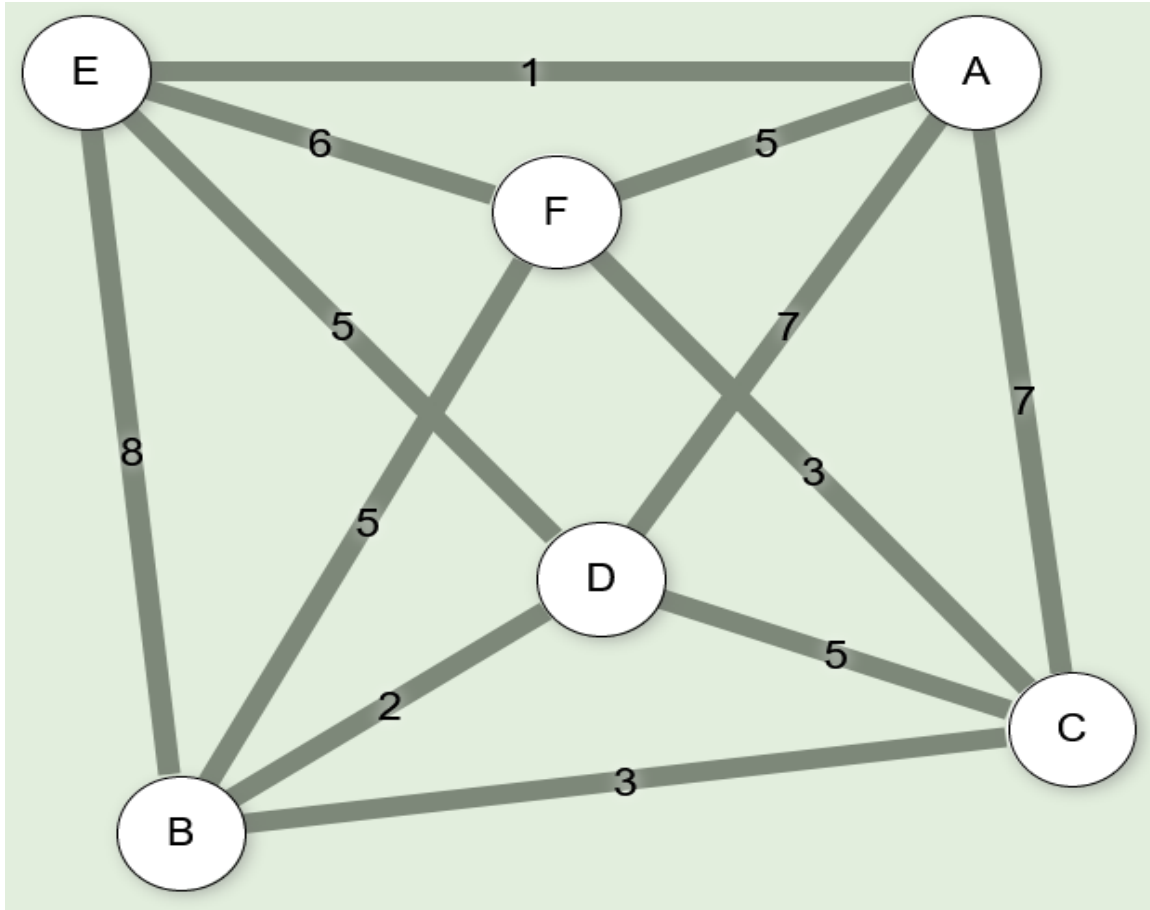


MST
 $3+5+2+3+7 = 20$



edge	weight
5-6	2
1-2	3
3-6	3
1-5	5
2-3	5
2-5	6
4-6	7
3-4	9

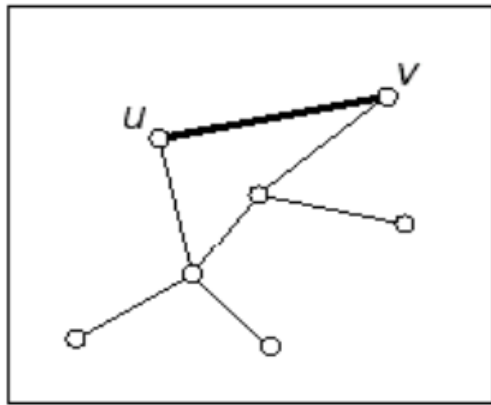
แบบฝึกหัด



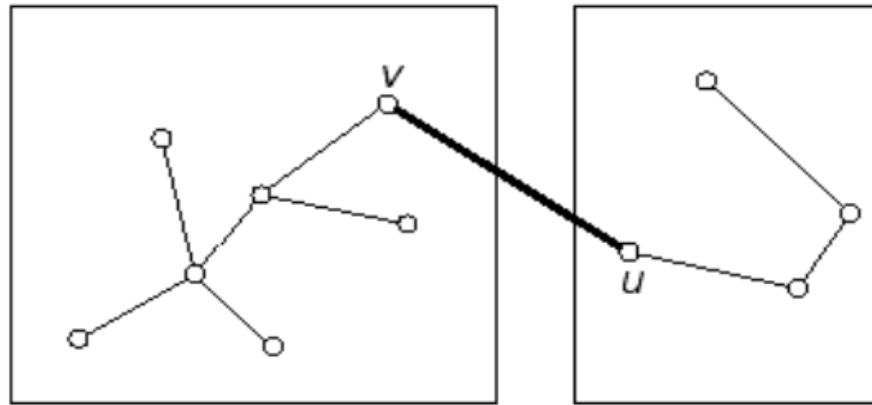
จงแสดงขั้นตอนการค้นหาต้นไม้แผ่ทั่วที่
เล็กที่สุดในกราฟ ดังต่อไปนี้ โดยใช้วิธี
Kruskal

การตรวจสอบวงวน

- พิจารณาว่าแต่ละเวอร์เท็กซ์ในกราฟเป็นต้นไม้ย่อย (sub-tree)
- ทุกครั้งที่เพิ่มเอดจ์ $e(u, v)$ ต้องเช็คกว่า u และ v อยู่ในต้นไม้ย่อยเดียวกันหรือไม่



(a)

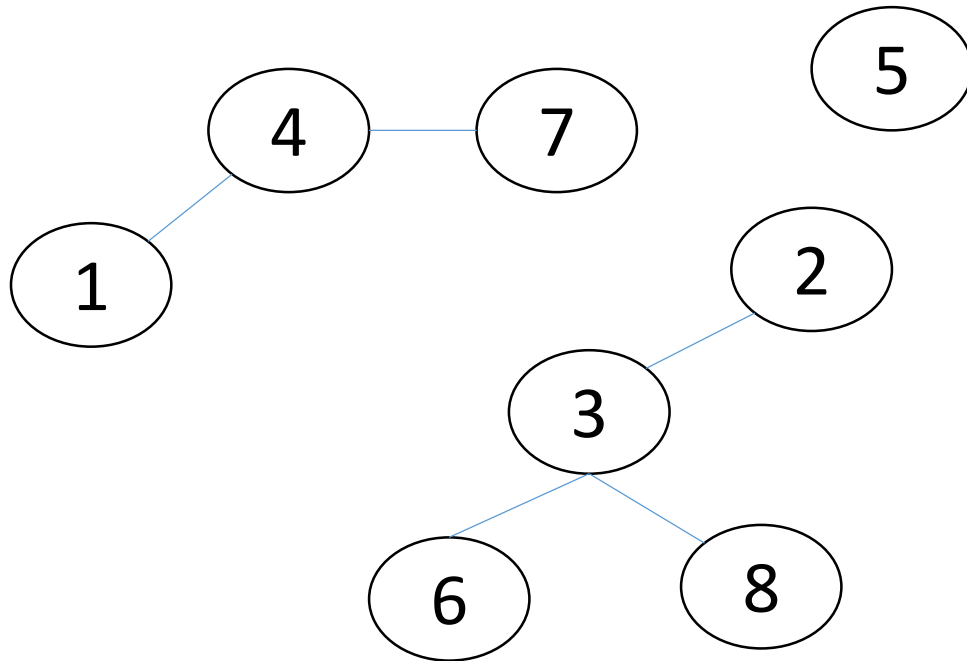


(b)

- ต้องการโครงสร้างข้อมูลที่ใช้ตรวจสอบว่าเวอร์เท็กซ์ใดอยู่ในต้นไม้ย่อยใด ซึ่งก็คือ disjoint set

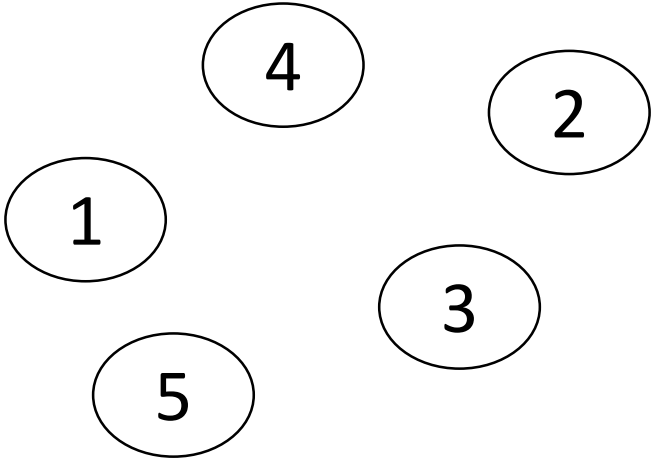
โครงสร้างข้อมูล disjoint sets

- ในทางปฏิบัติสามารถจัดเก็บต้นไม้ย่อย (subtree) ที่เกิดจากการเพิ่ม edge ใน kruskal โดยการใช้อาร์เรย์ parent-child โดยจะต้องกำหนดให้ต้นไม้ย่อยแต่ละต้นมีตัวแทน หรือ root node

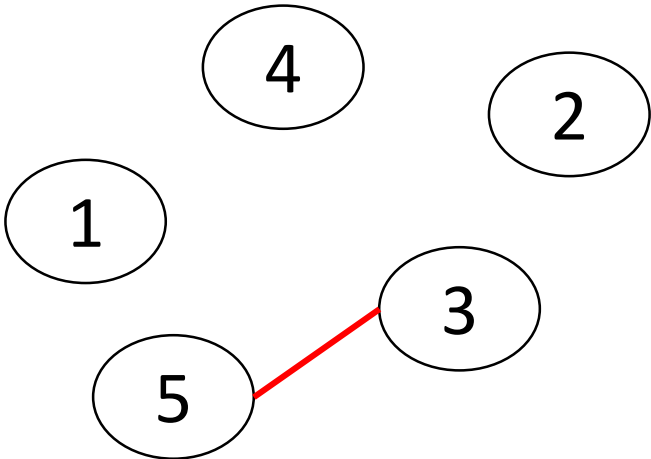


	1	2	3	4	5	6	7	8
parent	4	-1	2	-1	-1	3	4	3

ตัวอย่าง

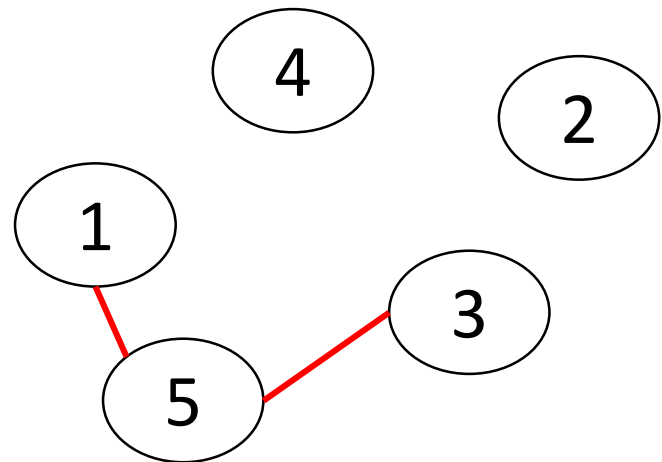


1	2	3	4	5
-1	-1	-1	-1	-1

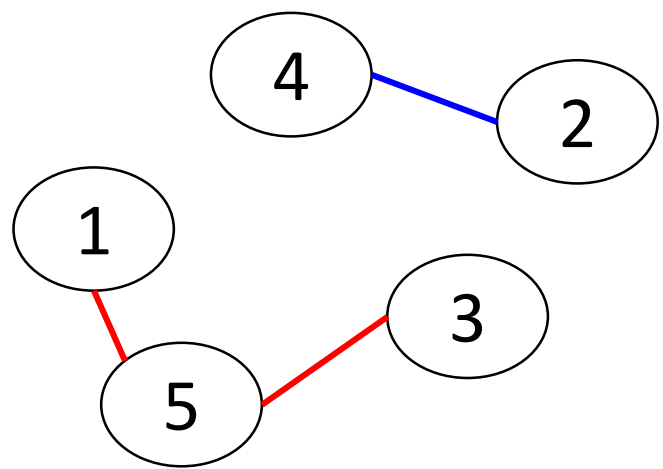


1	2	3	4	5
-1	-1	-1	-1	3

ตัวอย่าง



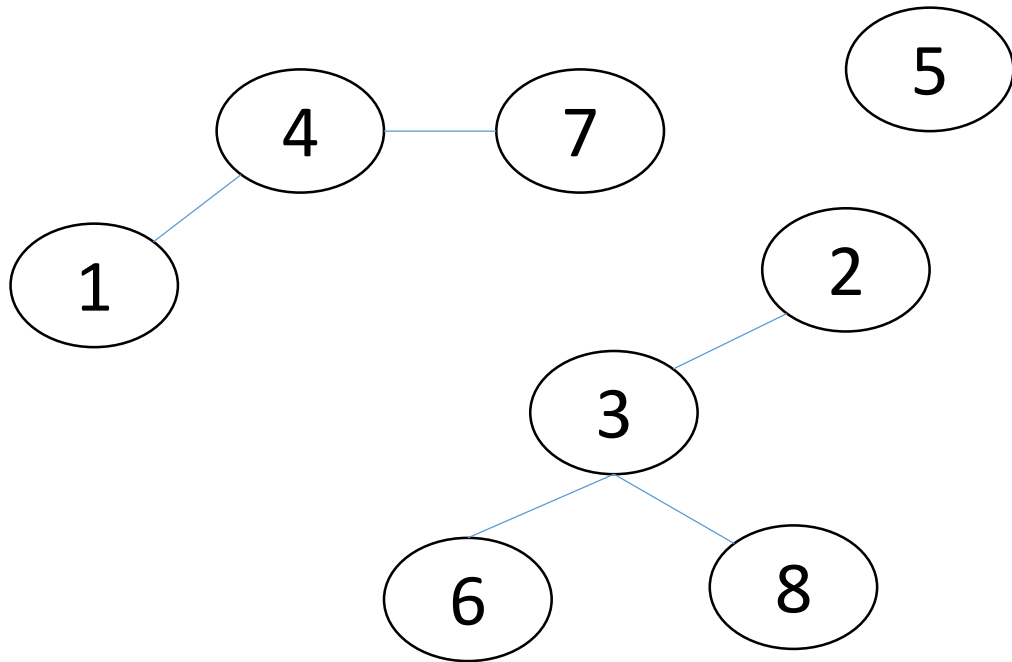
1	2	3	4	5
5	-1	-1	-1	3



1	2	3	4	5
5	-1	-1	2	3

โอเปอเรชัน find(i)

- ทำหน้าที่หาตัวแทนของต้นไม้ย่อยที่มีสมาชิกเป็น i



```
int find(int parent[], int i)
{ if(parent[i] == -1)
    return i;
  return find(parent, parent[i]);
}
```

	1	2	3	4	5	6	7	8
parent	4	-1	2	-1	-1	3	4	3

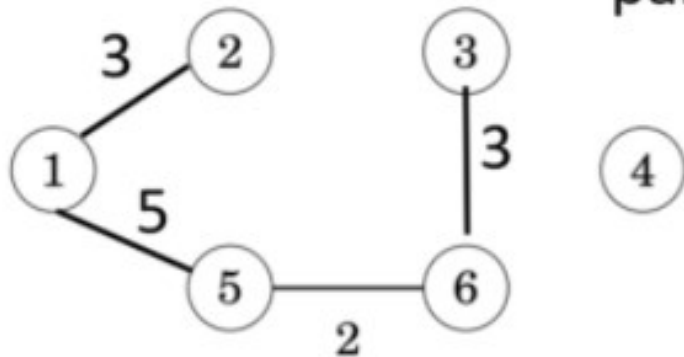
find(parent, 8) = ??

การตรวจสอบวงวนใน kruskal ด้วย find()

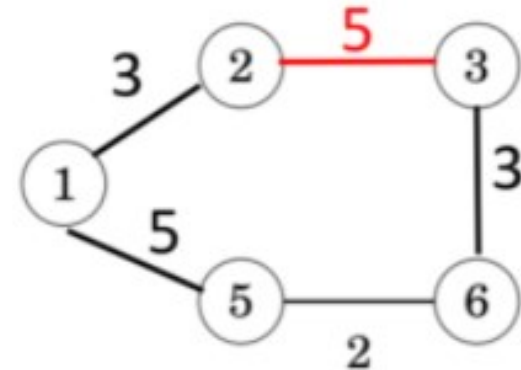
- จากการทำงานของ find(i) เราสามารถประยุกต์เพื่อตรวจสอบวงวนในต้นไม้ย่อยทุกครั้งที่มีการเพิ่มเอดจ์ (u, v) เข้าไป โดยตรวจสอบว่า find(u) และ find(v) มีตัวแทนเดียวกันหรือไม่

- สมมติให้ เอดจ์ (2, 3) ถูกเลือกเป็นลำดับถัดไป
- is_cycle(2, 3) = ???

```
int is_cycle(int u, int v)
{
    x = find(parent, u);
    y = find(parent, v);
    return (x==y)? true:false;
}
```



1	2	3	4	5	6
2	-1	6	-1	1	5

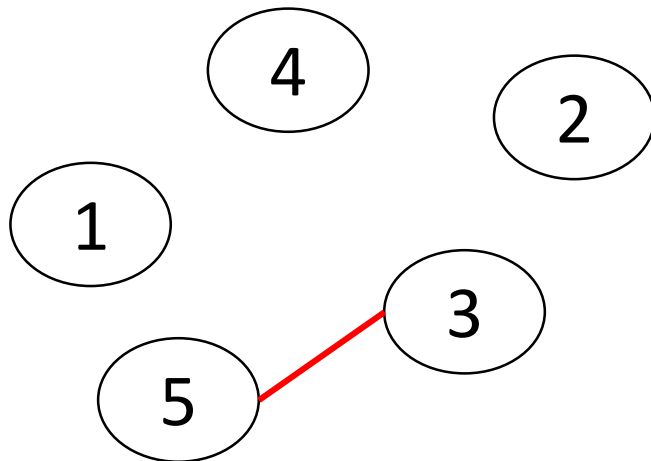
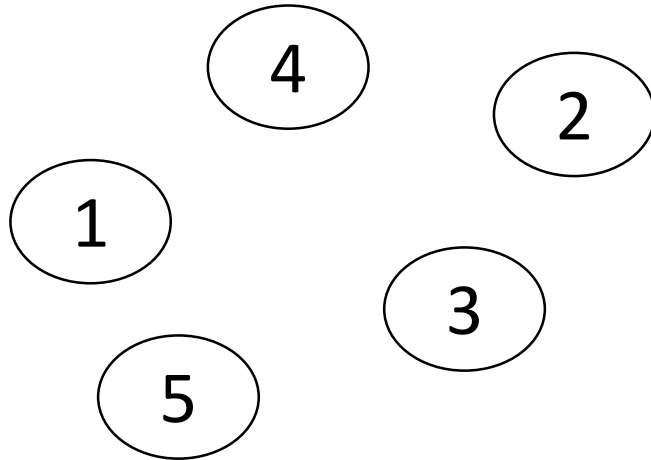


โอเปอเรชัน union(x, y)

- ทำหน้าที่รวมต้นไม้ย่อยที่มีสมาชิก x และต้นไม้ย่อยที่มีสมาชิก y เข้าด้วยกันกลายเป็นต้นไม้ย่อยเดียวกัน เมื่อตรวจสอบแล้วว่า edge (x, y) ไม่เกิดวงวน
- วิธีการคือทำการเปลี่ยนสถานะของตัวแทนของต้นไม้ย่อยฝั่งใดฝั่งหนึ่งในอาร์เรย์ parent ให้เป็น child ของตัวแทนในต้นไม้ย่อยอีกฝั่งหนึ่ง

```
void union(int parent[], int x, int y) {  
    int xroot = find(parent, x);  
    int yroot = find(parent, y);  
    parent[xroot] = yroot;  
}
```

ตัวอย่าง

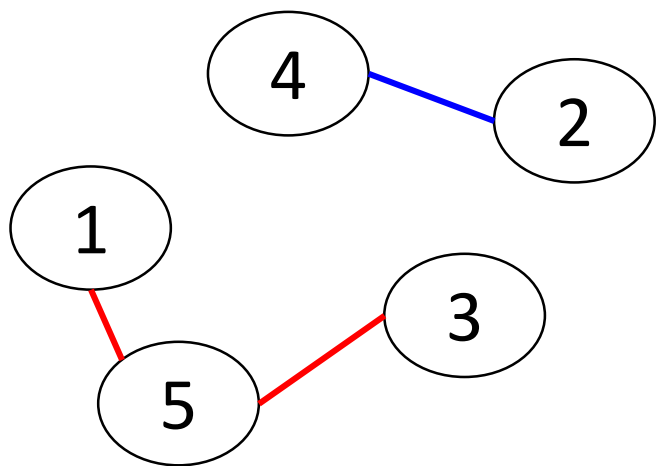
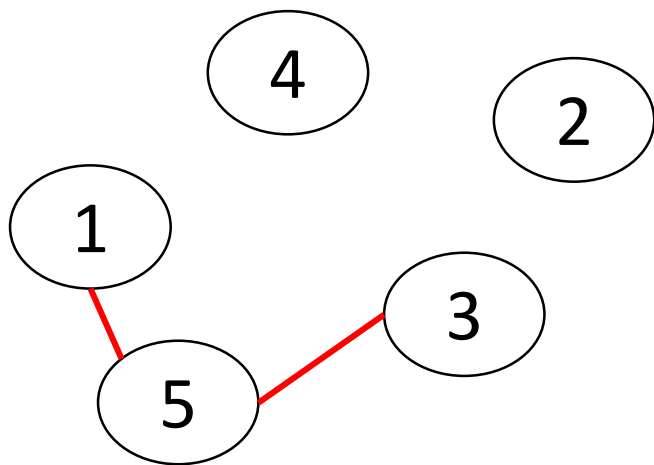


1	2	3	4	5
-1	-1	-1	-1	-1

สมมติจะเพิ่มเอดจ์ (5, 3) หาก (5, 3) ไม่ทำให้เกิดวงวน ก็จะทำกร
เพิ่มเอดจ์เข้าไปในต้นไม้ โดยการหาตัวแทน 5 และ 3
โดยเรียก find(5) และ find(3) จากนั้นนำตัวแทนที่ได้ไปเปลี่ยนสถานะ

1	2	3	4	5
-1	-1	-1	-1	3

ตัวอย่าง



เพิ่มเอดจ์ (1, 5) หาก (1, 5) ไม่ทำให้เกิดวงวน ก็จะทำให้การเพิ่มเอดจ์เข้าไปในต้นไม้
โดยค้นหาตัวแทนของ 1 และ 5 จากการเรียก

โดยเรียก $\text{find}(1) = 1$ และ $\text{find}(5) = 3$ จากนั้นนำตัวแทนที่ได้ไปเปลี่ยนสถานะ

1	2	3	4	5
3	-1	-1	-1	3

เพิ่มเอดจ์ (2, 4) หาก (2, 4) ไม่ทำให้เกิดวงวน ก็จะทำให้การเพิ่มเอดจ์เข้าไปในต้นไม้
โดยค้นหาตัวแทนของ 2 และ 4 จากการเรียก

โดยเรียก $\text{find}(2) = 2$ และ $\text{find}(4) = 4$ จากนั้นนำตัวแทนที่ได้ไปเปลี่ยนสถานะ

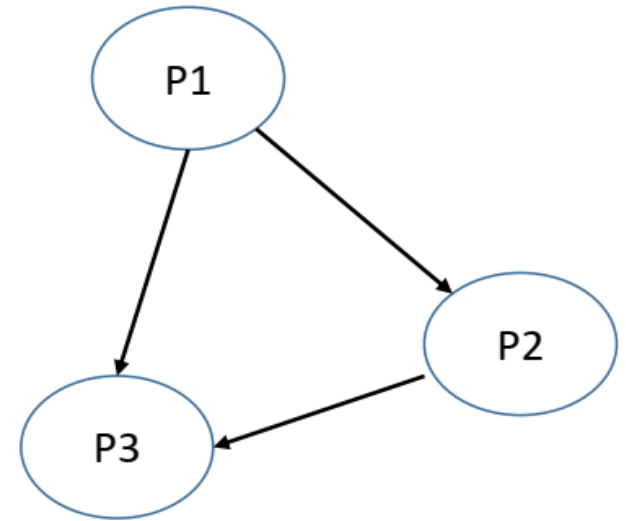
1	2	3	4	5
3	-1	-1	2	3

Kruskal's algorithm

```
int find(vector<int>& parent, int i) {  
    if (parent[i] == i)  
        return i;  
    return find(parent, parent[i]);  
}
```

```
void union1(vector<int>& parent, int x, int y) {  
    int xroot = find(parent, x);  
    int yroot = find(parent, y);  
    parent[xroot] = yroot;  
}
```

```
int is_cycle(vector<int>& parent, int u, int v)  
{  
    int x = find(parent, u);  
    int y = find(parent, v);  
    return (x==y)? true:false;  
}
```



```

void kruskal(vector<pair<int, int>> adj[], int V) {
    priority_queue< pair<int, pair<int, int>>, vector<pair<int, pair<int, int>>>, greater<pair<int,
pair<int, int>>> > pq;

    vector<int> parent(V);
    for (int i = 0; i < V; ++i)                // initial parent array
        parent[i] = i;

    for (int i = 0; i < V; i++) {                // sort edges by pushing into queue
        for(int j = 0; j < adj[i].size(); j++)
            pq.push({adj[i][j].second, {i, adj[i][j].first}});
    }

    int ne = 0                                // count selected edge
    while (ne < V-1) {
        int u = pq.top().second.first;
        int v = pq.top().second.second;
        int w = pq.top().first;
        pq.pop();

        if(!is_cycle(parent, u, v)) {            // check cycle when adding (u, v)
            ne++;
            union1(parent, u, v);
            cout << "(" << u << "," << v << "," << w << ")"<<endl;
        }
    }
}

```

Kruskal's algorithm